Lecture 4: Unsupervised learning

Alex Reinhart

Machine Learning II: 46-927

February 15–17, 2021

Announcements

- Submit your final project teams and topics by this Friday, through Gradescope ("Project Proposal").
- Everyone should have a team now—if not, let me know ASAP

Today:

- Short discussion of the quiz
- Final project
- Unsupervised learning
  Returning to Clustering and Dimension reduction in more detail.
  - Clustering
  - Dimension reduction?

Review so far

In clustering we divide up our data points into groups or clusters.

One goal could be minimizing within-cluster scatter, itself an infeasible problem. K-means approximates this solution iteratively for Euclidean distance and fixed K. Depends on the random initialization.

The K-medoids algorithm solves a similar approximate problem for general dissimilarity, while constraining the centers to be observed points. This can give more interpretable cluster centers.

From K-means to hierarchical clustering

Recall two properties of K-means (K-medoids) clustering:

1. Final clustering depends on the initial centers (does not find global max)
2. Fits exactly K clusters (for fixed K)

Suppose we cluster with $K = 3$, and then decide we want a more refined clustering with $K = 4$. The clusterings may have no relation. This makes tuning the level of clustering a bit odd.

Hierarchical clustering will produce a result that:

- Is deterministic, based on the distances $d_{ij} = d(x_i, x_j)$. *No random starts.*
- Describes the resulting clusterings across all levels of clustering (numbers of clusters K)
- Gives nested clusters as K changes!

Demonstration of hierarchical clustering

[hier_demo.R]

Agglomerative vs divisive

Two types of hierarchical clustering algorithms

Agglomerative (i.e., bottom-up):

- Start with all points in their own cluster
- Until there is only one cluster, repeatedly: merge the two groups that have the smallest dissimilarity

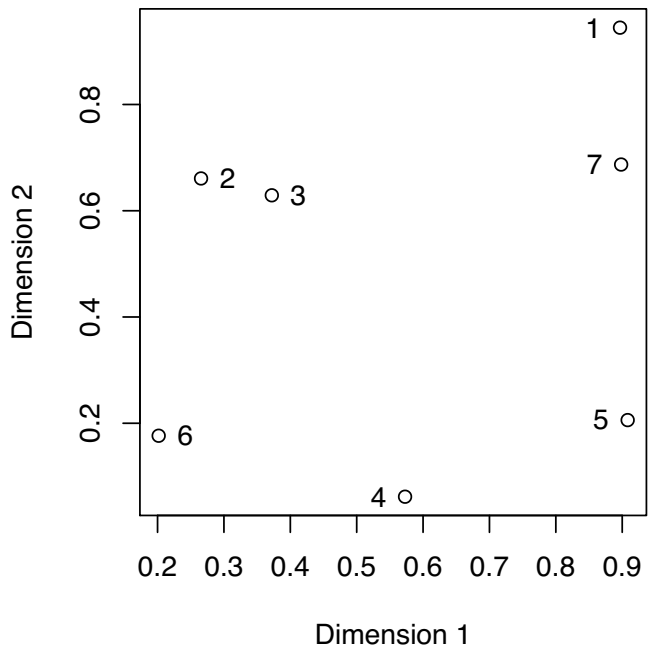
Divisive (i.e., top-down):

- Start with all points in one cluster
- Until all points are in their own cluster, repeatedly: split the group into two resulting in the biggest dissimilarity

Agglomerative strategies are simpler and most common, so we will focus on those.

Simple example

Given these data points, an agglomerative algorithm might decide on a clustering sequence as follows:



Step 1: $\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}$;

Step 2: $\{1\}, \{2, 3\}, \{4\}, \{5\}, \{6\}, \{7\}$;

Step 3: $\{1, 7\}, \{2, 3\}, \{4\}, \{5\}, \{6\}$;

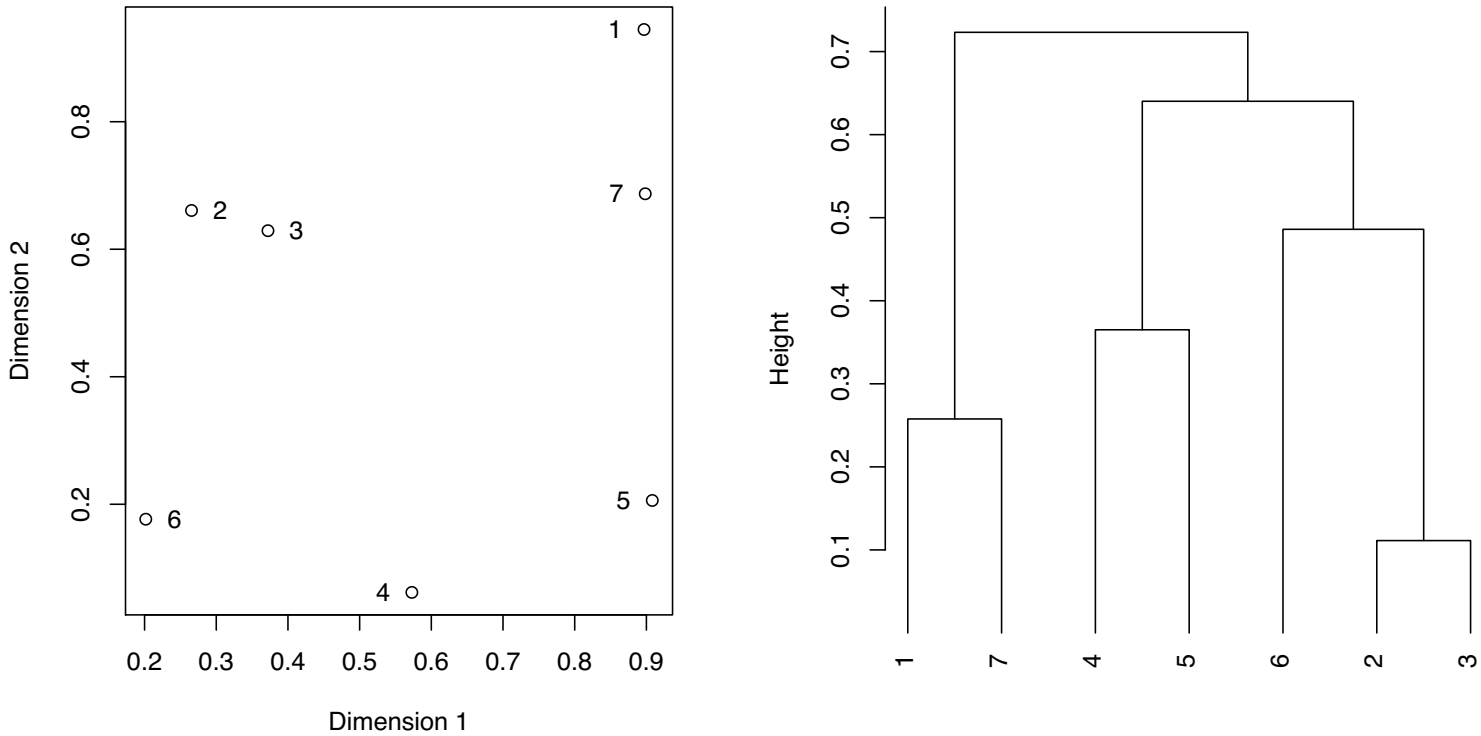
Step 4: $\{1, 7\}, \{2, 3\}, \{4, 5\}, \{6\}$;

Step 5: $\{1, 7\}, \{2, 3, 6\}, \{4, 5\}$;

Step 6: $\{1, 7\}, \{2, 3, 4, 5, 6\}$;

Step 7: $\{1, 2, 3, 4, 5, 6, 7\}$.

We can also represent the sequence of clustering assignments as a dendrogram:



Note that cutting the dendrogram horizontally partitions the data points into clusters

What's a dendrogram?

Dendrogram: convenient graphic to display a hierarchical sequence of clustering assignments. This is simply a tree where:

- Each node represents a group
- Each leaf node is a singleton (i.e., a group containing a single data point)
- Root node is the group containing the whole data set
- Each internal node has two children, representing the the groups that were merged to form it

If we fix the leaf nodes at height zero, then each internal node is drawn at a height proportional to the dissmilarity between its two daughter nodes

Linkages

We started with distances between points, d_{ij}.

However, you will have noticed that we need a distance between groups to carry out agglomerative clustering.

The distance between groups is called a linkage.

The choice of linkage dramatically impacts the behavior of a clustering method. Different linkages lead to different interpretations and guarantees.

Linkages

Given points $X_1, \ldots, X_n$, and dissimilarities d_{ij} between each pair X_i and X_j. (Think of $X_i \in \mathbb{R}^p$ and $d_{ij} = \|X_i - X_j\|_2$; note: this is distance, not squared distance)

At any level, clustering assignments can be expressed by sets $G = \{i_1, i_2, \ldots, i_r\}$, giving indices of points in this group. Let n_G be the size of G (here $n_G = r$). Bottom level: each group looks like $G = \{i\}$, top level: only one group, $G = \{1, \ldots, n\}$

Linkage: function $d(G, H)$ that takes two groups G, H and returns a dissimilarity score between them

Agglomerative clustering, given the linkage:

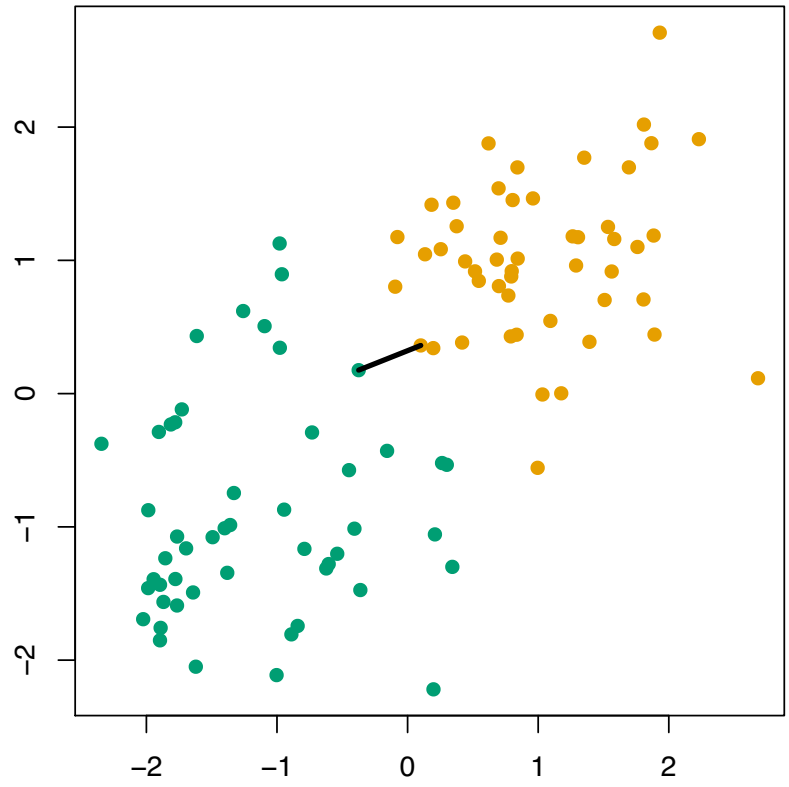
- Start with all points in their own group
- Until there is only one cluster, repeatedly: merge the two groups G, H such that $d(G, H)$ is smallest

Single linkage

In single linkage (i.e., nearest-neighbor linkage), the dissimilarity between G, H is the smallest dissimilarity between two points in opposite groups:

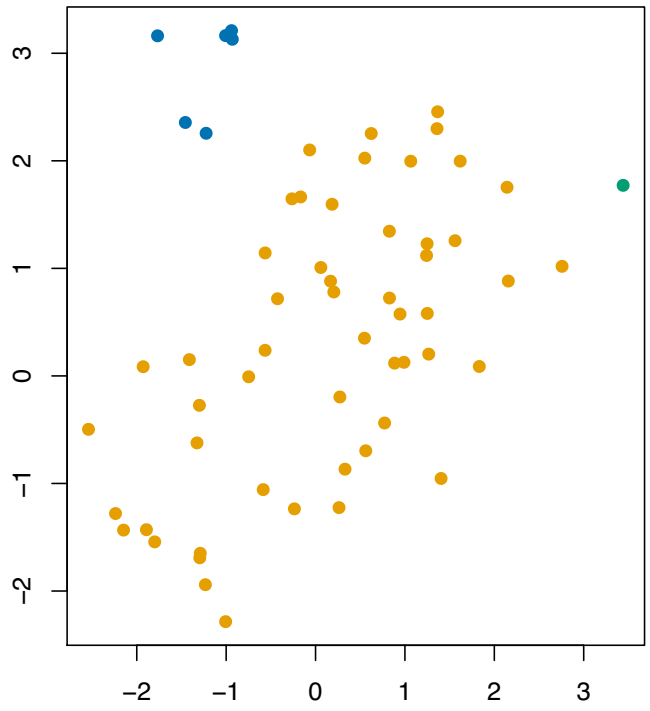
$$d_{\text{single}}(G, H) = \min_{i \in G,\, j \in H} d_{ij}$$

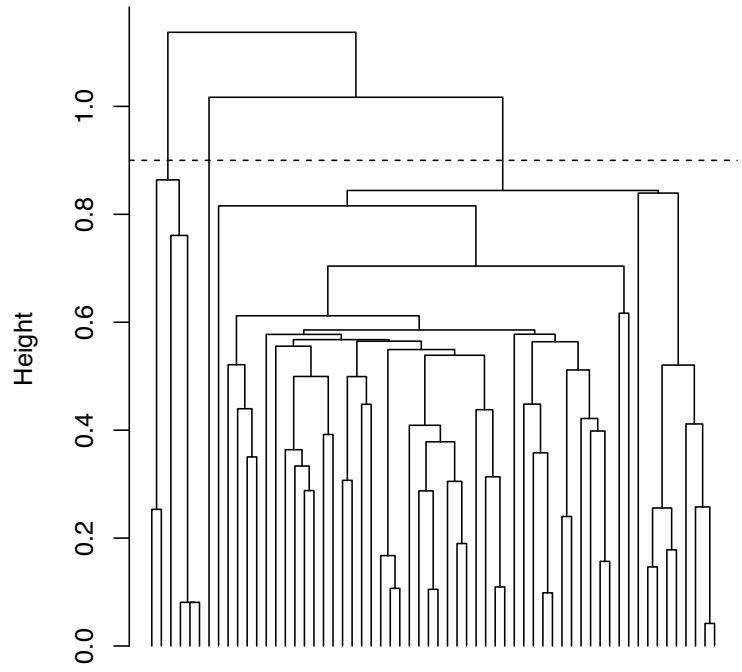
Example (dissimilarities d_{ij} are distances, groups are marked by colors): single linkage score $d_{\text{single}}(G, H)$ is the distance of the closest pair



Single linkage example

Here $n = 60$, $X_i \in \mathbb{R}^2$, $d_{ij} = \|X_i - X_j\|_2$. Cutting the tree at $h = 0.9$ gives the clustering assignments marked by colors





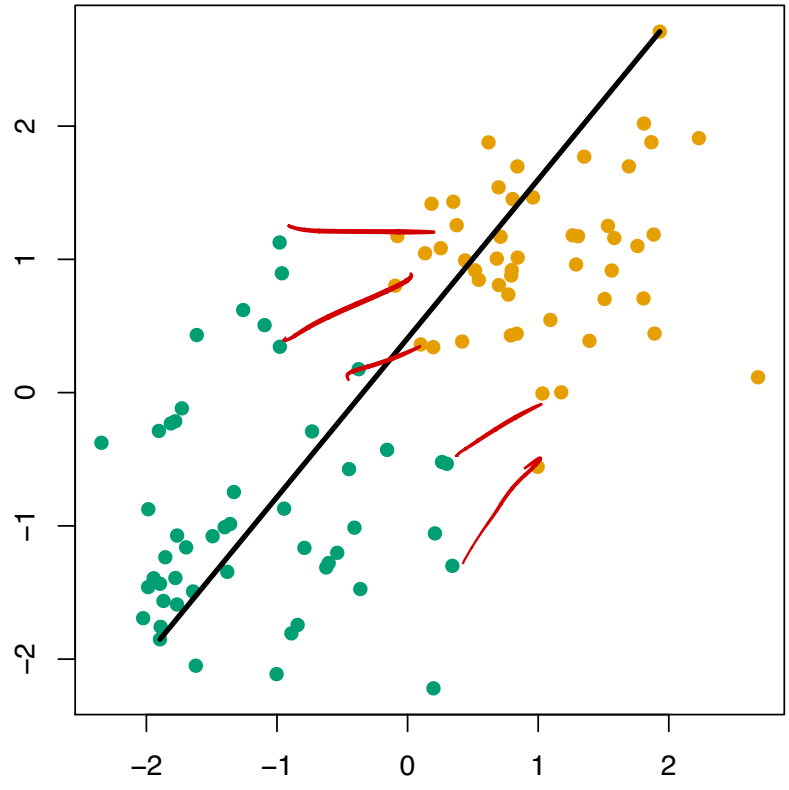
Cut interpretation: If X_i and X_j are in different groups, $d_{ij} > 0.9$.

Complete linkage

In complete linkage (i.e., furthest-neighbor linkage), dissimilarity between G, H is the largest dissimilarity between two points in opposite groups:

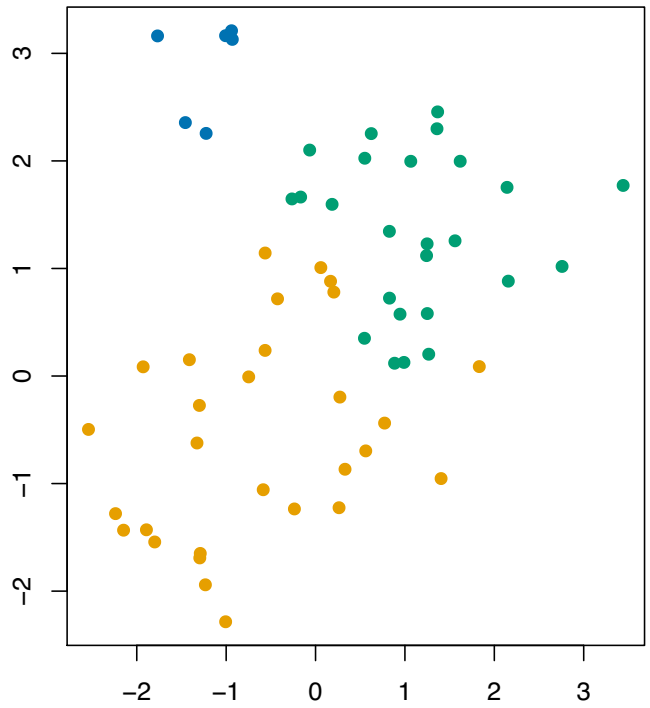
$$d_{\text{complete}}(G, H) = \max_{i \in G,\, j \in H} d_{ij}$$

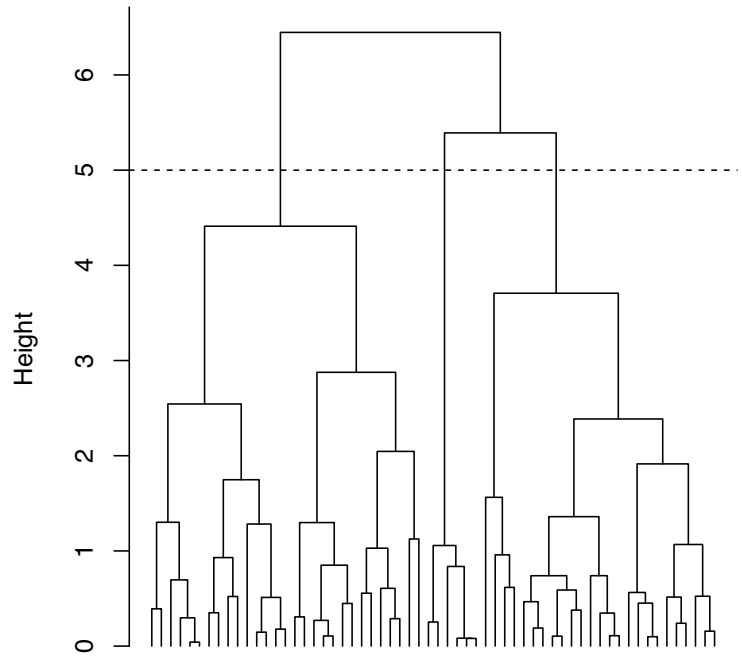
Example (dissimilarities d_{ij} are distances, groups are marked by colors): complete linkage score $d_{\text{complete}}(G, H)$ is the distance of the furthest pair



Complete linkage example

Same data as before. Cutting the tree at $h = 5$ gives the clustering assignments marked by colors





Cut interpretation: for each point X_i, every other point X_j in its cluster satisfies $d_{ij} \leq 5$

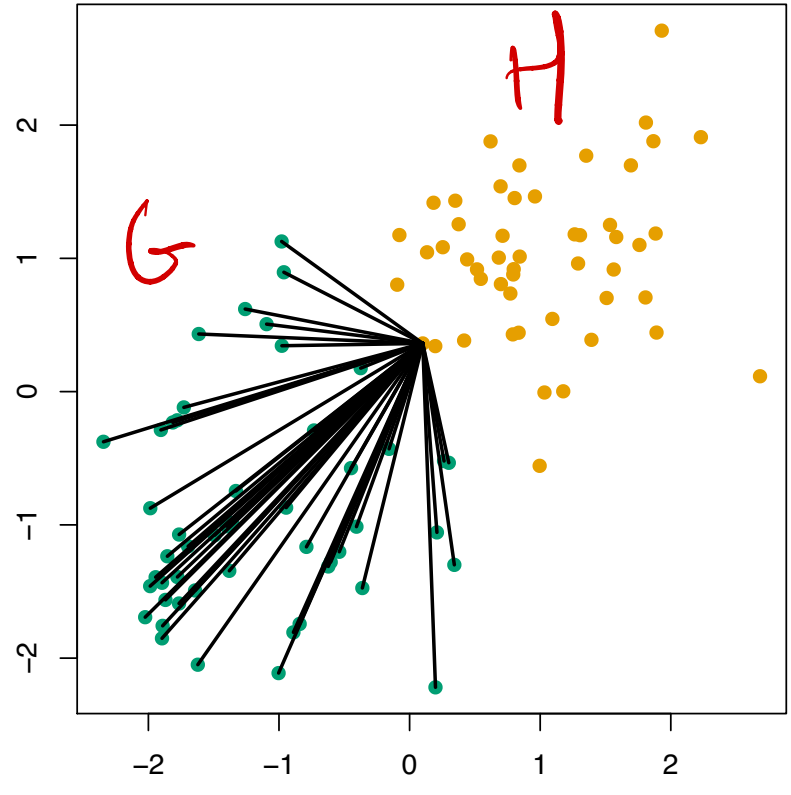
Average linkage

In average linkage, the dissimilarity between G, H is the average dissimilarity over all points in opposite groups:

$$d_{\text{average}}(G, H) = \frac{1}{n_G \cdot n_H} \sum_{i \in G,\, j \in H} d_{ij}$$

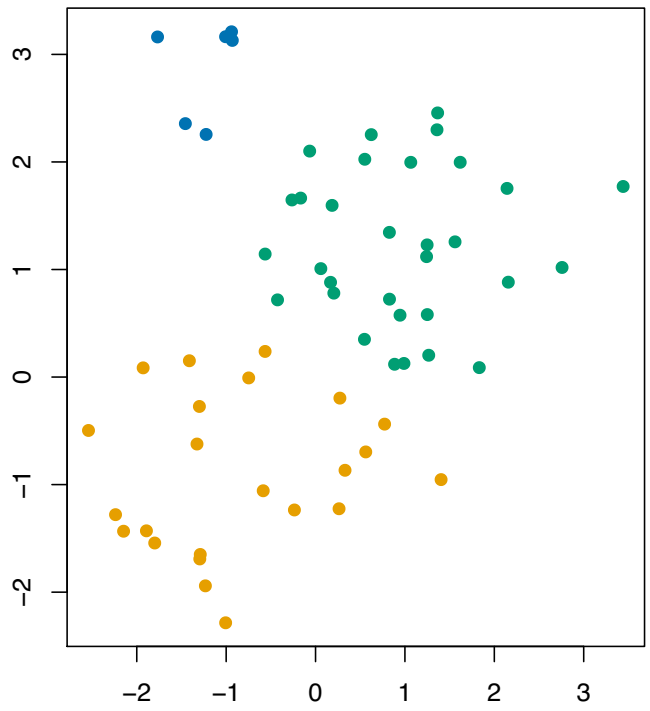
Example (dissimilarities d_{ij} are distances, groups are marked by colors): average linkage score $d_{\text{average}}(G, H)$ is the average distance across all pairs

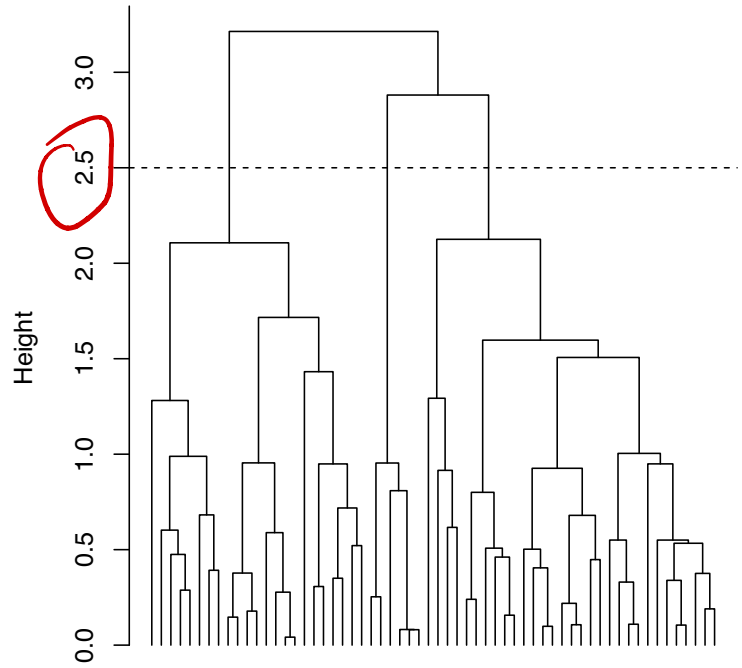
(Plot here only shows distances between the ~~blue~~ green points and one ~~red~~ yellow point)



Average linkage example

Same data as before. Cutting the tree at $h = 1.5$ gives clustering assignments marked by the colors





Cut interpretation: there really isn't a good one!

Common properties

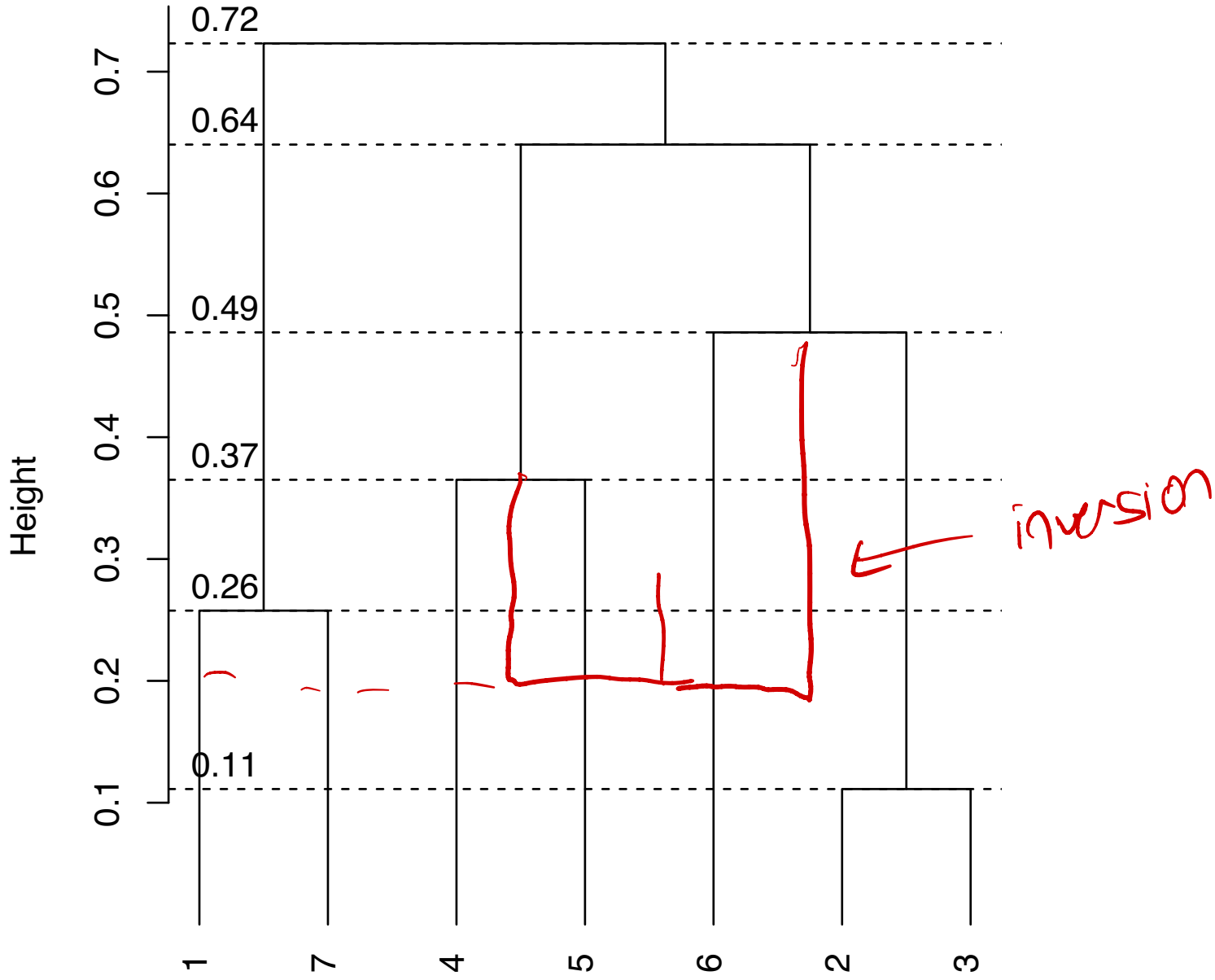
Single, complete, average linkage share the following properties:

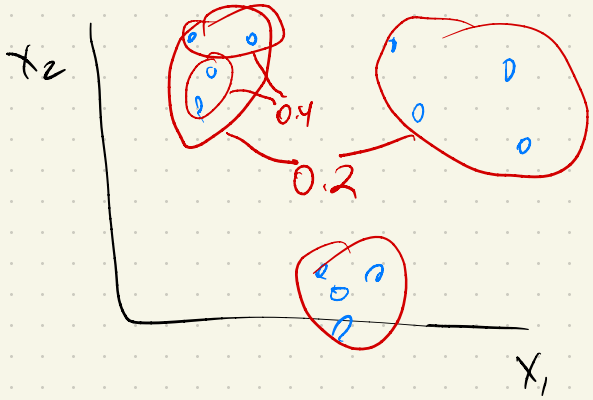
- These linkages operate on dissimilarities d_{ij}, and don't need the points $X_1, \ldots, X_n$ to be in Euclidean space
- Running agglomerative clustering with any of these linkages produces a dendrogram with no inversions

Second property, in words: disimilarity scores between merged clusters only increase as we run the algorithm

Means that we can draw a proper dendrogram, where the height of a parent is always higher than height of its daughters

Example of a dendrogram with no inversions




x_2
0.4
0.2
x_1

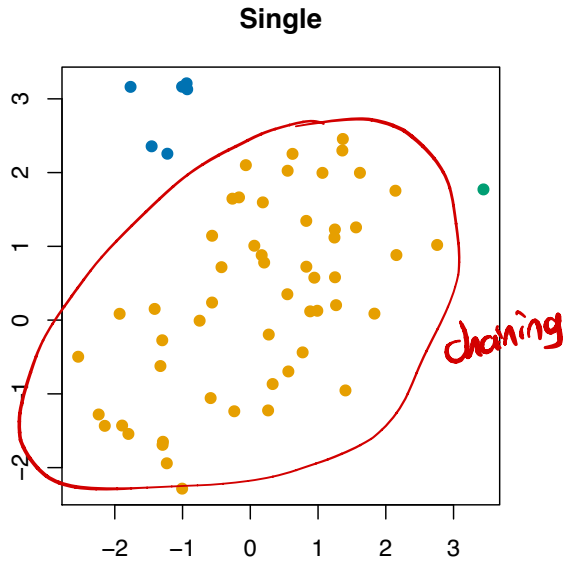
Shortcomings of single, complete linkage

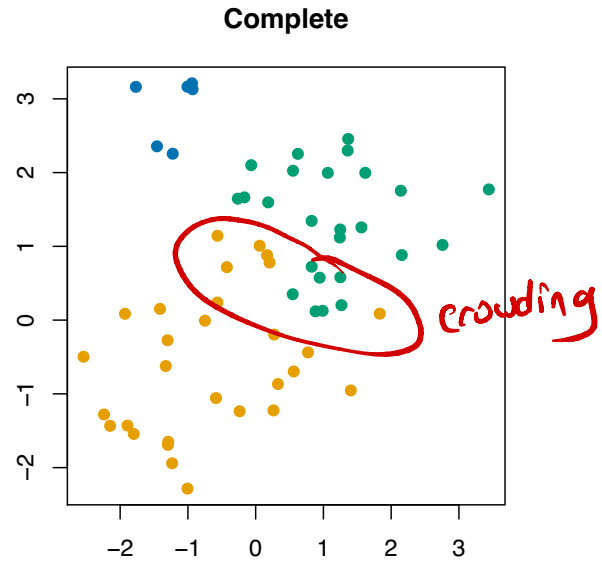
Single and complete linkage can have some practical problems:

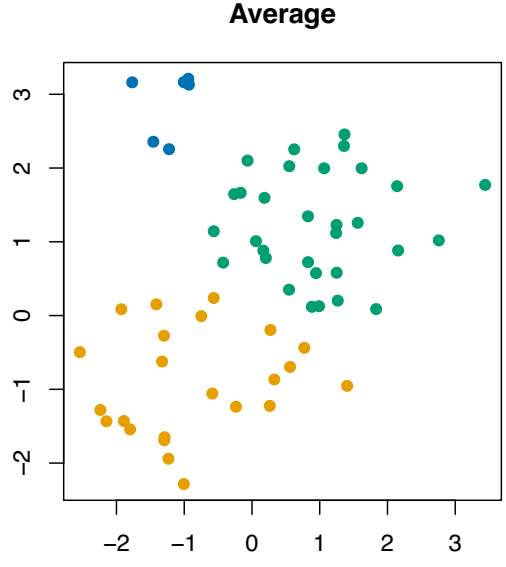
- Single linkage suffers from chaining. In order to merge two groups, only need one pair of points to be close, irrespective of all others. Therefore clusters can be too spread out, and not compact enough
- Complete linkage avoids chaining, but suffers from crowding. Because its score is based on the worst-case dissimilarity between pairs, a point can be closer to points in other clusters than to points in its own cluster. Clusters are compact, but not far enough apart

Average linkage tries to strike a balance. It uses average pairwise dissimilarity, so clusters tend to be relatively compact and relatively far apart

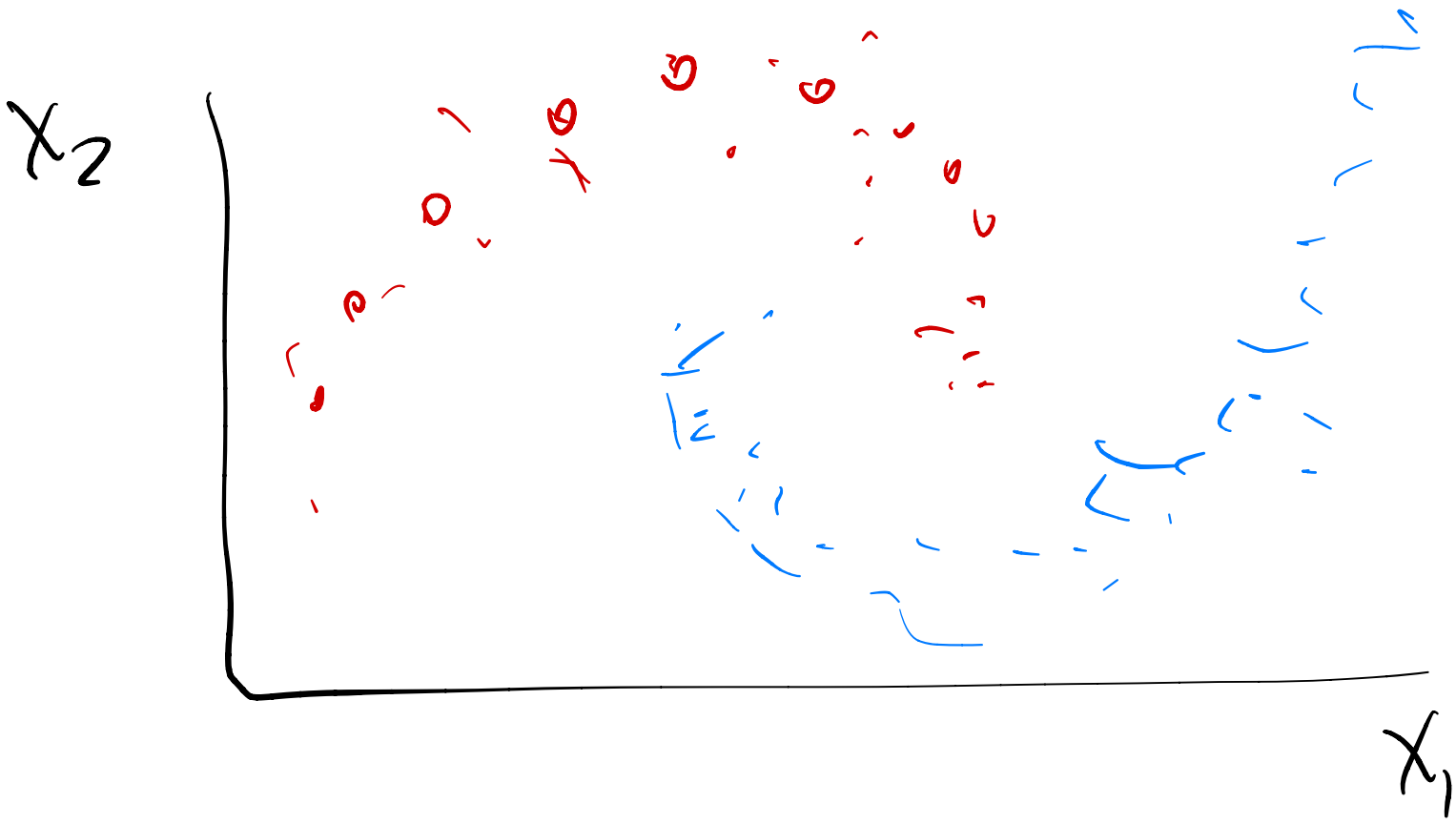
Example of chaining and crowding







Sometimes chaining can be good!



Shortcomings of average linkage

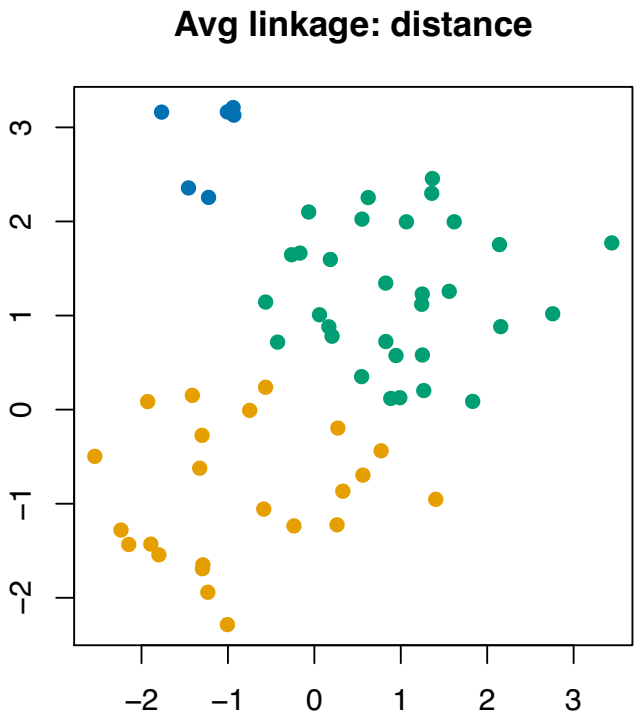
Average linkage isn't perfect, it has its own problems:

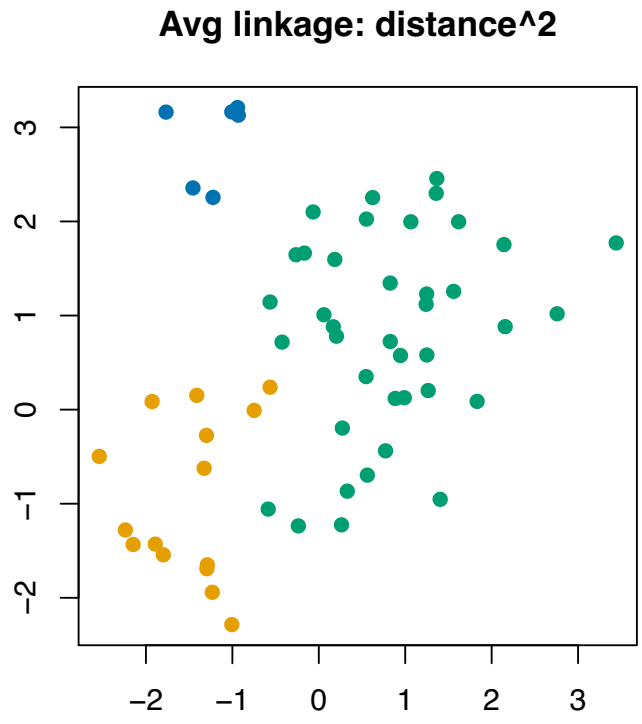
- It is not clear what properties the resulting clusters have when we cut an average linkage tree at given height h. Single and complete linkage trees each had simple interpretations

- Results of average linkage clustering can change with a monotone increasing transformation of dissimilarities d_{ij}. I.e., if h is such that $h(x) \leq h(y)$ whenever $x \leq y$, and we used dissimilarites $h(d_{ij})$ instead of d_{ij}, then we could get different answers

Note: results of single, complete linkage clustering are unchanged under monotone transformations b/c they use min and max

Example of a change with monotone increasing transformation



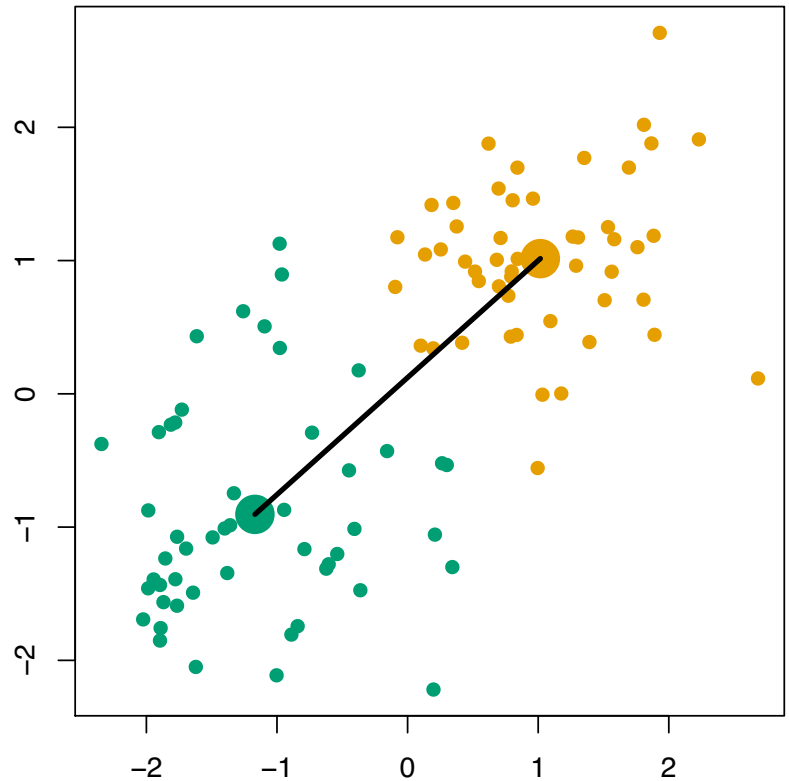


Centroid linkage

Centroid linkage[1] is commonly used. Assume that $X_i \in \mathbb{R}^p$, and $d_{ij} = \|X_i - X_j\|_2$. Let $\bar{X}_G, \bar{X}_H$ denote group averages for G, H. Then:

$$d_{\text{centroid}}(G, H) = \|\bar{X}_G - \bar{X}_H\|_2$$

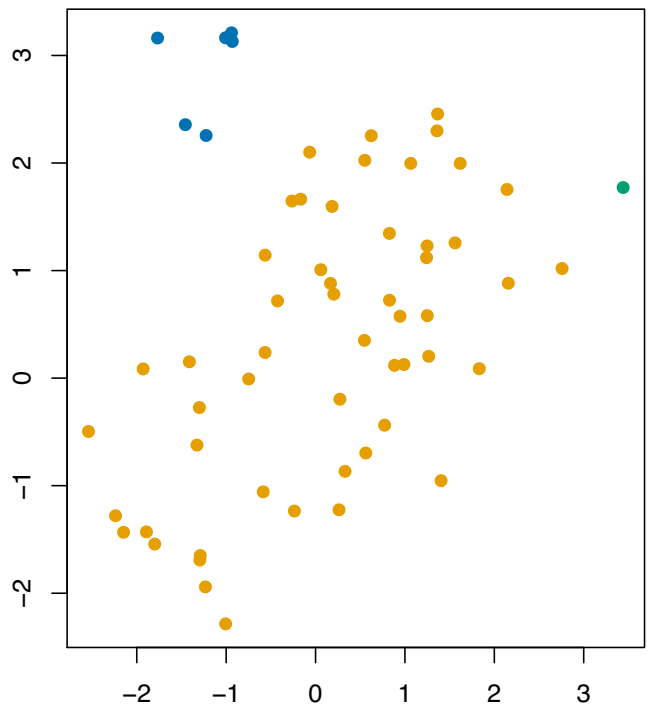
Example (dissimilarities d_{ij} are distances, groups are marked by colors): centroid linkage score $d_{\text{centroid}}(G, H)$ is the distance between the group centroids (i.e., group averages)

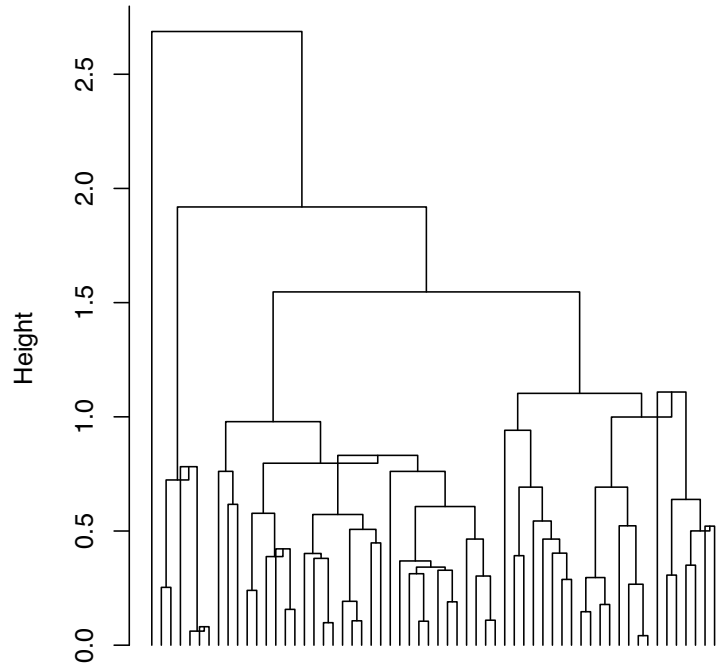


[1] Eisen et al. (1998), "Cluster Analysis and Display of Genome-Wide Expression Patterns"

Centroid linkage example

Here $n = 60$, $X_i \in \mathbb{R}^2$, $d_{ij} = \|X_i - X_j\|_2$. Cutting the tree at some heights wouldn't make sense ... because the dendrogram has inversions! But we can, e.g., still look at output with 3 clusters

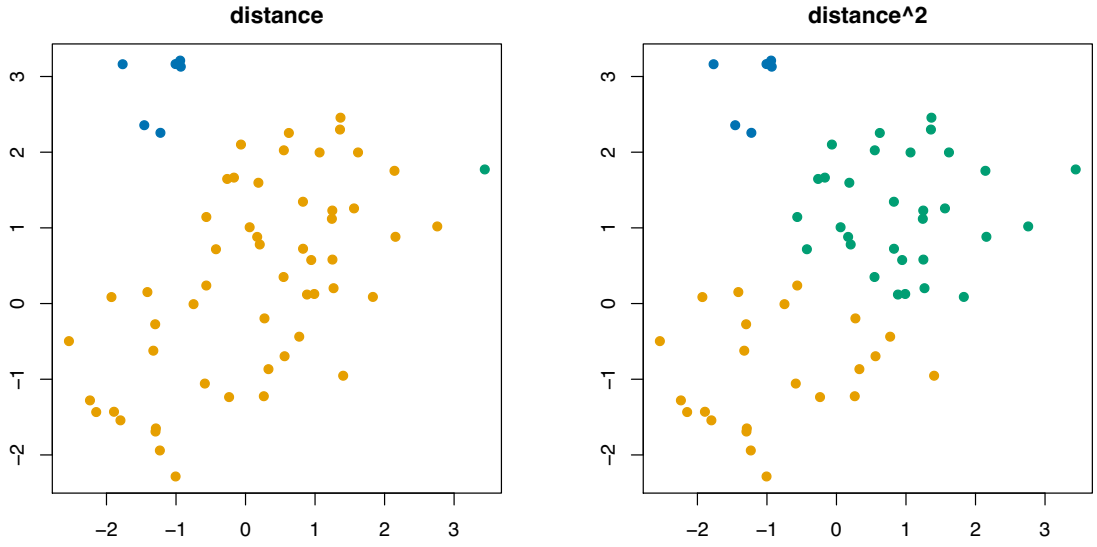




Cut interpretation: there isn't one, even with no inversions

Shortcomings of centroid linkage

- Can produce dendrograms with inversions, which really messes up the visualization
- Even if were we lucky enough to have no inversions, still no interpretation for the clusters resulting from cutting the tree
- Answers change with a monotone transformation of the dissimilarity measure $d_{ij} = \|X_i - X_j\|_2$. E.g., changing to $d_{ij} = \|X_i - X_j\|_2^2$ would give a different clustering

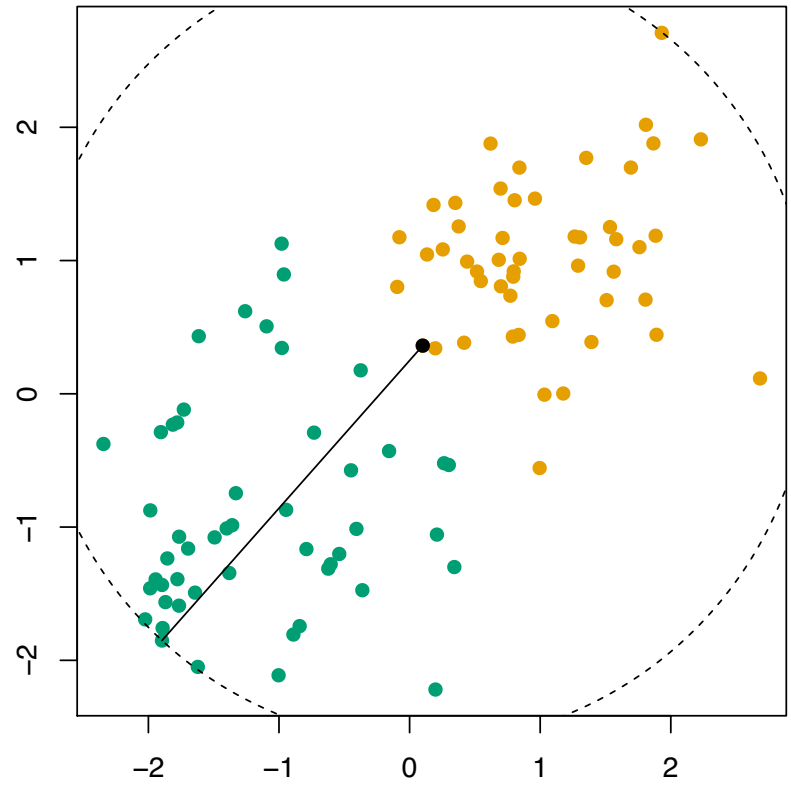


Minimax linkage

Minimax linkage[2] is a newcomer. First define radius of a group of points G around X_i as $r(X_i, G) = \max_{j \in G} d_{ij}$. Then:

$$d_{\text{minimax}}(G, H) = \min_{i \in G \cup H} r(X_i, G \cup H)$$

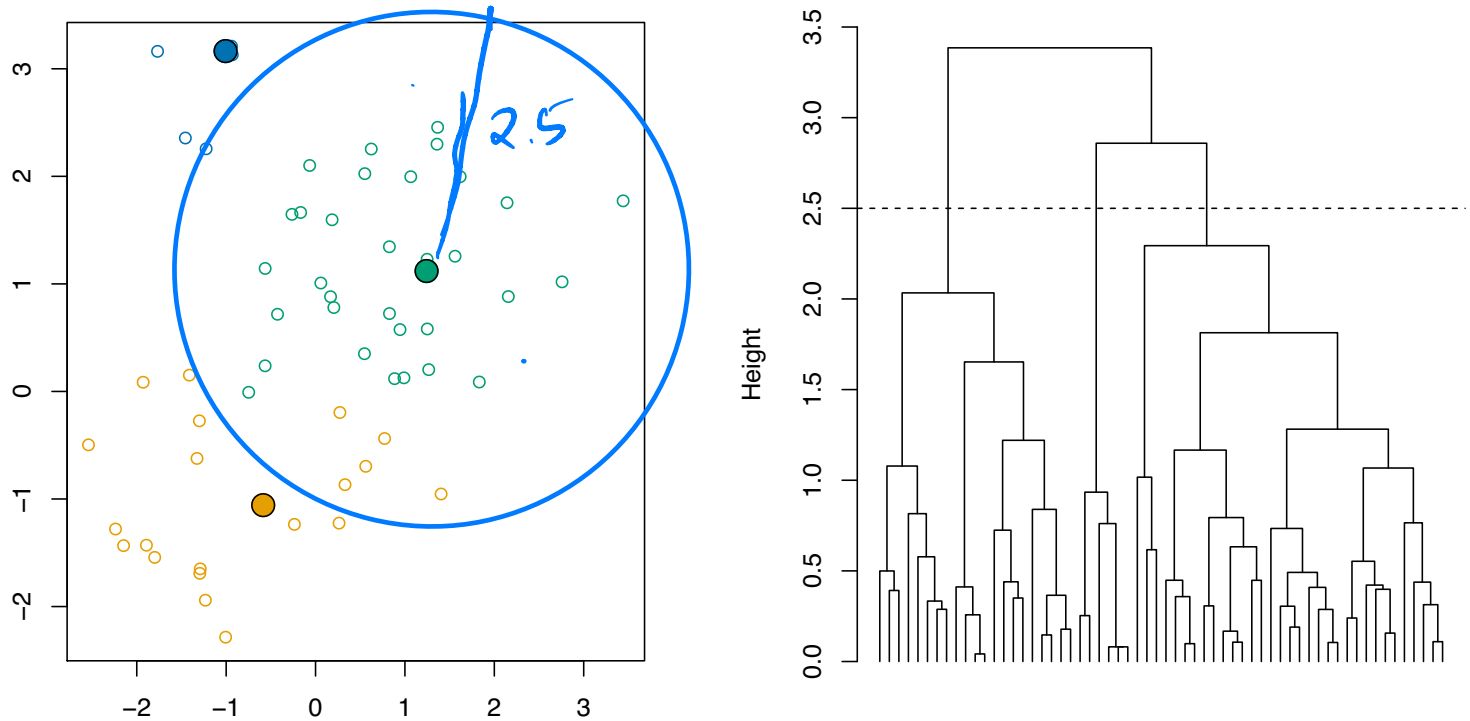
Example (dissimilarities d_{ij} are distances, groups marked by colors): minimax linkage score $d_{\text{minimax}}(G, H)$ is the smallest radius encompassing all points in G and H. The center X_c is the black point



[2] Bien et al. (2011), "Hierarchical Clustering with Prototypes via Minimax Linkage"

Minimax linkage example

Same data as before. Cutting the tree at $h = 2.5$ gives clustering assignments marked by the colors

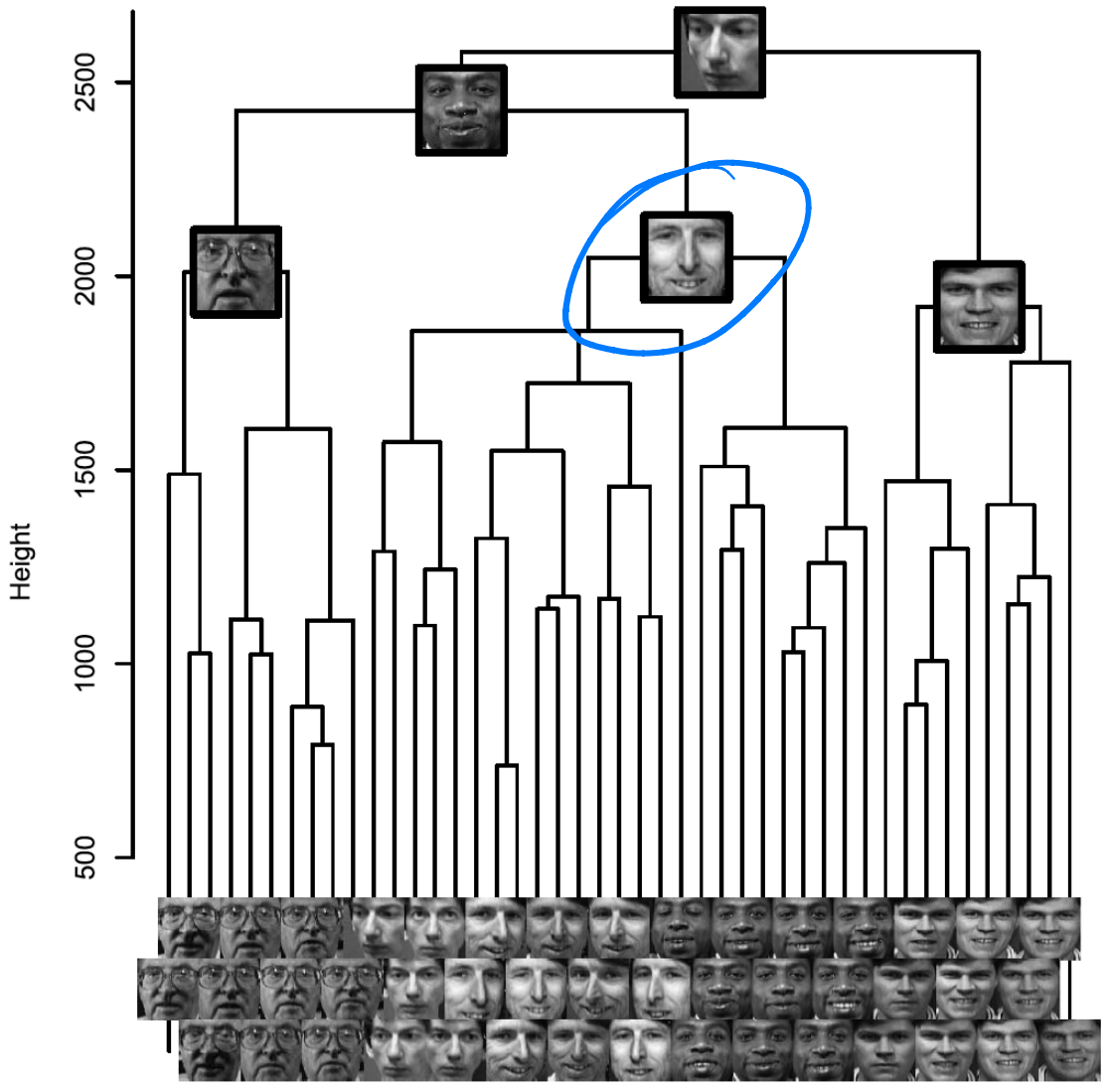


Cut interpretation: each point X_i belongs to a cluster whose center X_c satisfies $d_{ic} \leq 2.5$

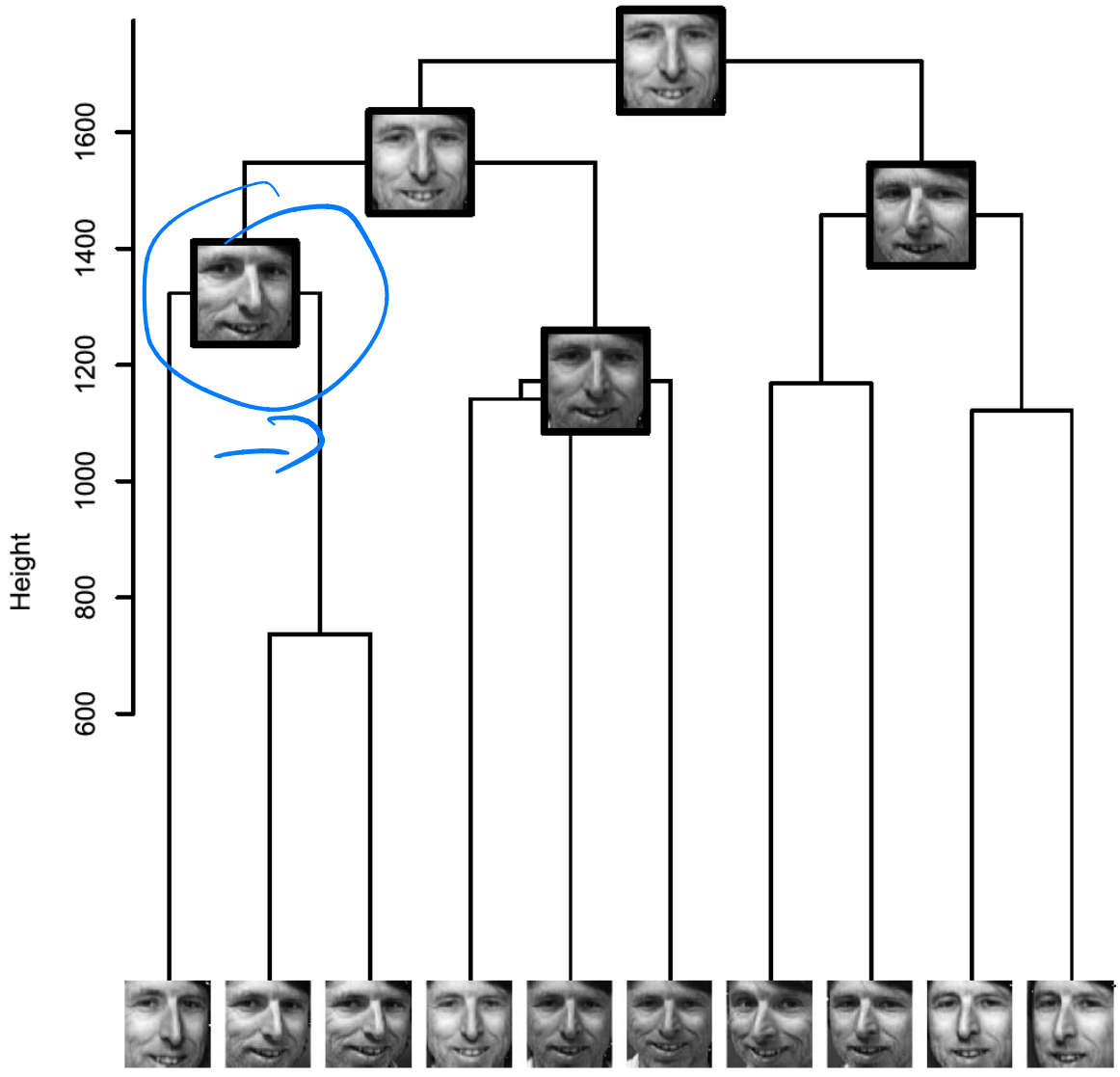
Properties of minimax linkage

- Cutting a minimax tree at a height h a nice interpretation: each point is $\leq h$ in dissimilarity to the center of its cluster. (This is related to a famous set cover problem)
- Produces dendrograms with no inversions
- Unchanged by monotone transformation of dissimilarities d_{ij}
- Produces clusters whose centers are chosen among the data points themselves. Remember that, depending on the application, this can be a very important property. (Hence minimax clustering is the analogy to K-medoids in the world of hierarchical clustering)

Example: Olivetti faces dataset



(From Bien et al. (2011))



(From Bien et al. (2011))

Linkages summary

Linkage	No inversions?	Unchanged with monotone transformation?	Cut interpretation?	Notes
Single	✓	✓	✓	chaining
Complete	✓	✓	✓	crowding
Average	✓	×	×	
Centroid	×	×	×	simple
Minimax	✓	✓	✓	centers are data points

This still doesn't say what the best linkage is for a given problem.

It's turns out to be very situation dependent. You need to see what works well for your purpose. However, these ideas can give you an intuition of where to start and what can go wrong.

Finding groups of highly similar stocks

Suppose we have a bunch of assets, and a similarity score between each pair (e.g., correlation). I want to find groups of stocks that are all tightly correlated with one another, so that I can identify potential arbitrage opportunities.

How can we use hierarchical clustering, and with what linkage?

complete linkage to guarantee similarity within clusters

Identifying dissimilar assets

Suppose we have a bunch of assets, and a similarity score between each pair (e.g., correlation). I want to find groups of stocks so that all stocks in one group are guaranteed to have low correlation with all stocks from any other group. This will help me pick a diverse portfolio.

How can we use hierarchical clustering, and with what linkage?

Single linkage

How many clusters?

Sometimes, using K-means, K-medoids, or hierarchical clustering, we might have no problem specifying the number of clusters K ahead of time, e.g.,

- Segmenting a client database into K clusters for K salesmen
- Compressing an image using vector quantization, where K controls the compression rate

Other times, K is implicitly defined by cutting a hierarchical clustering tree at a given height, e.g., I know I want groups of stock with correlation at least ρ within group.

But in most exploratory applications, the number of clusters K is unknown. So we are left asking the question: what is the "right" value of K?

This is a hard problem

Determining the number of clusters is a hard problem!

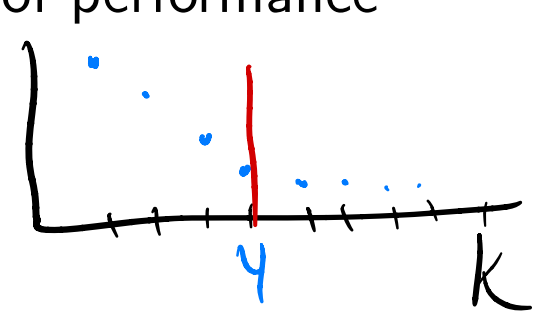
Why is it hard?

- There are no labels!
- The "right" number of clusters depends on what you're going to use clusters for! There's no pre-defined right answer!
- Even in simple settings, determining the number of clusters is a hard task for humans to perform (unless the data are low-dimensional). Not only that, it's just as hard to explain what it is we're looking for.

There is no very satisfying answer

Some approaches that have been tried:

- Looking for the "elbow" in some measure of performance (e.g., within or between group scatter)



- Looking at the ratio (CH index)

between

$$CH(K) = \frac{B(K)/(K-1)}{W(K)/(n-K)}$$

within

- Comparing to within group scatter of uniformly distributed points (gap statistic, Tibshirani et. al., 2001)

- Squint at dendrograms

Reminder: within-cluster variation

We're going to focus on K-means, but most ideas will carry over to other settings (like hierarchical)

Recall: given the number of clusters K, the K-means algorithm approximately minimizes the within-cluster variation:

$$W(K) = \sum_{k=1}^{K} \sum_{C(i)=k} \|X_i - \bar{X}_k\|_2^2$$

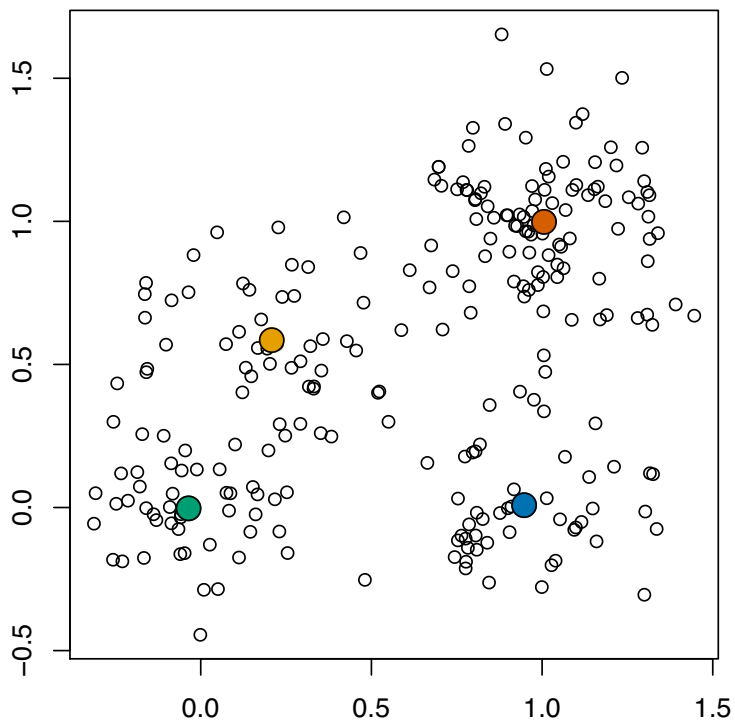
over clustering assignments C, where $\bar{X}_k$ is the average of points in group k, $\bar{X}_k = \frac{1}{n_k} \sum_{C(i)=k} X_i$

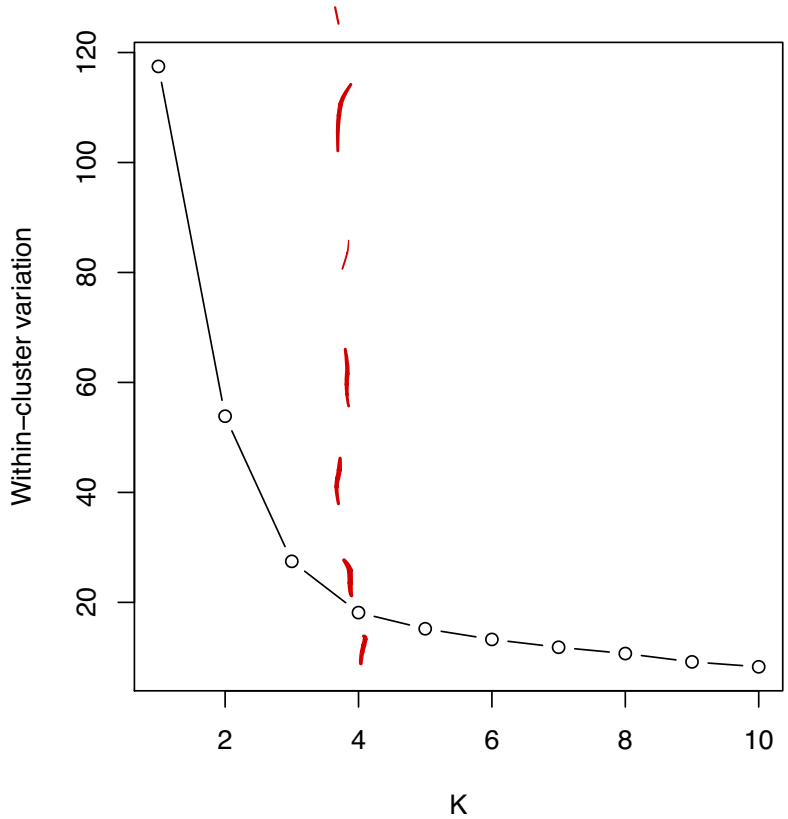
Clearly a lower value of W is better. So why not just run K-means for a bunch of different values of K, and choose the value of K that gives the smallest $W(K)$?

That's not going to work

Problem: within-cluster variation just keeps decreasing

Example: $n = 250$, $p = 2$, $K = 1, \ldots, 10$





Between-cluster variation

Within-cluster variation measures how tightly grouped the clusters are. As we increase the number of clusters K, this just keeps going down. What are we missing?

How about: Between-cluster variation measures how spread apart the groups are from each other:

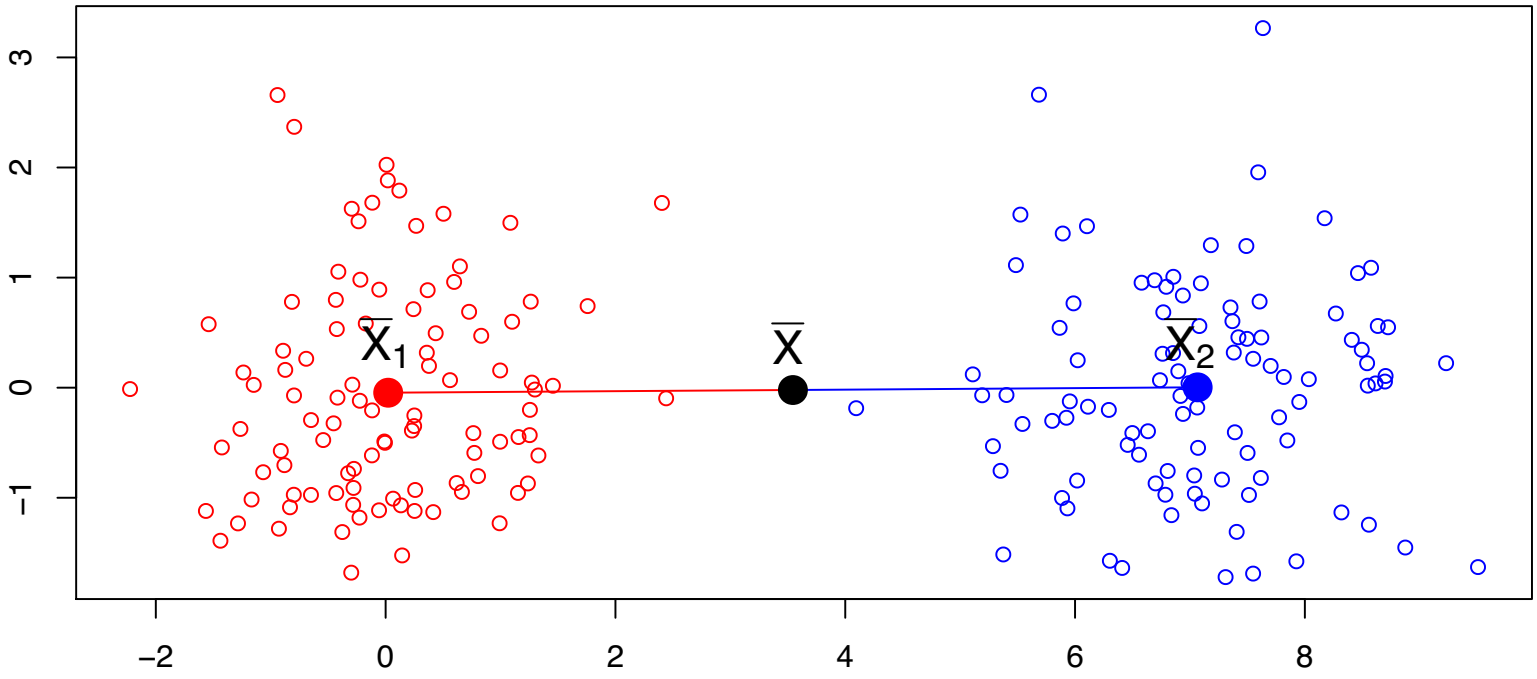
$$B(K) = \sum_{k=1}^{K} n_k \|\bar{X}_k - \bar{X}\|_2^2$$

where as before $\bar{X}_k$ is the average of points in group k, and $\bar{X}$ is the overall average, i.e.

$$\bar{X}_k = \frac{1}{n_k} \sum_{C(i)=k} X_i \quad \text{and} \quad \bar{X} = \frac{1}{n} \sum_{i=1}^{n} X_i$$

Example: between-cluster variation

Example: $n = 100$, $p = 2$, $K = 2$



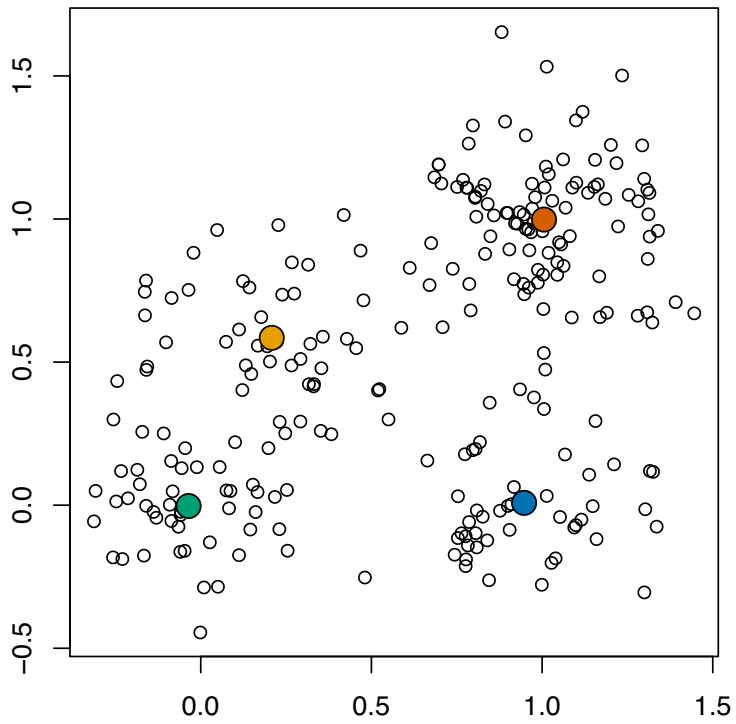
$$B(2) = n_1 \|\bar{X}_1 - \bar{X}\|_2^2 + n_2 \|\bar{X}_2 - \bar{X}\|_2^2$$

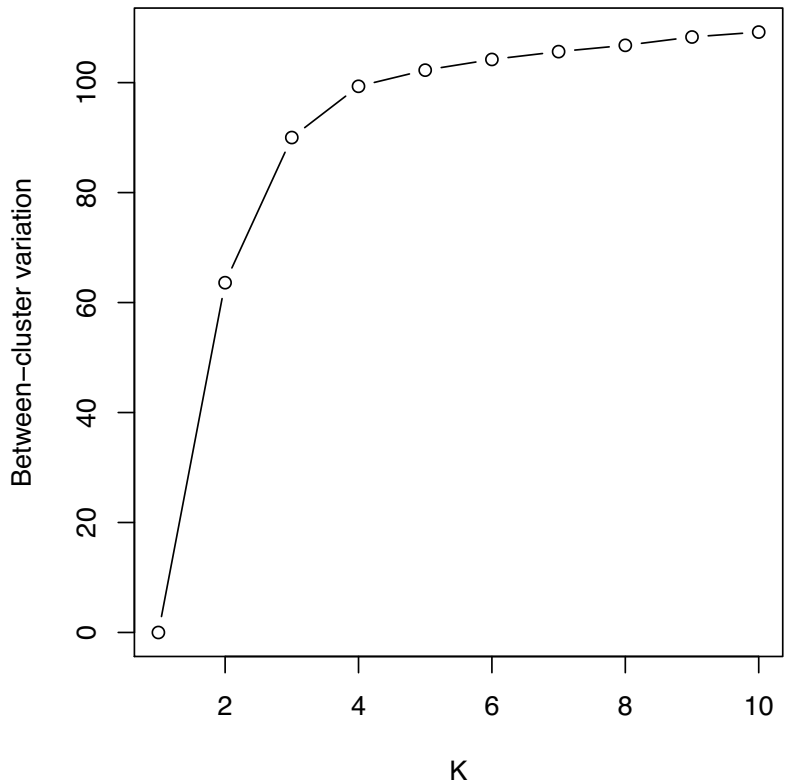
$$W(2) = \sum_{C(i)=1} \|X_i - \bar{X}_1\|_2^2 + \sum_{C(i)=2} \|X_i - \bar{X}_2\|_2^2$$

Still not going to work

Bigger B is better, can we use it to choose K? Problem: between-cluster variation just keeps increasing

Running example: $n = 250$, $p = 2$, $K = 1, \ldots, 10$





CH index

Ideally we'd like our clustering assignments C to simultaneously have a small W and a large B

This is the idea behind the CH index.[3] For clustering assignments coming from K clusters, we record CH score:

$$\mathsf{CH}(K) = \frac{B(K)/(K-1)}{W(K)/(n-K)}$$

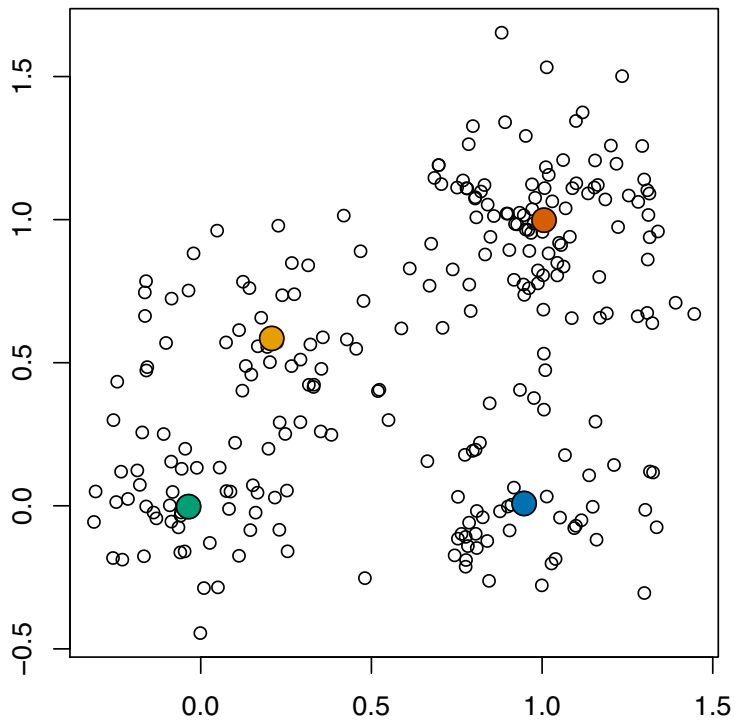
To choose K, just pick some maximum number of clusters to be considered K_{max} (e.g., $K = 20$), and choose the value of K with the largest score $\mathsf{CH}(K)$, i.e.,

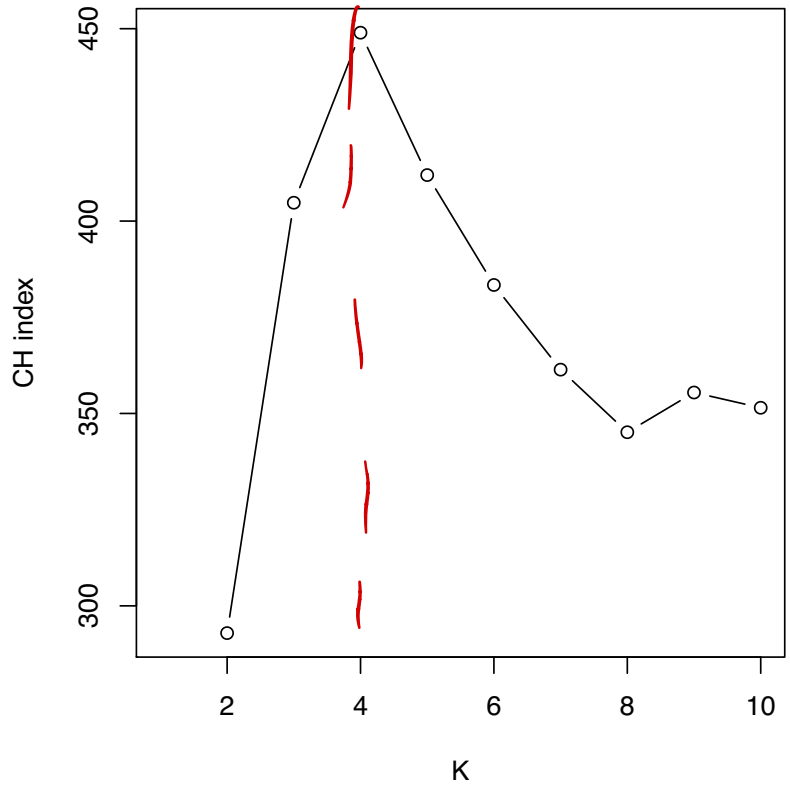
$$\hat{K} = \underset{K \in \{2,\ldots,K_{\mathsf{max}}\}}{\operatorname{argmax}} \mathsf{CH}(K)$$

[3] Calinski and Harabasz (1974), "A dendrite method for cluster analysis"

Example: CH index

Running example: $n = 250$, $p = 2$, $K = 2, \ldots, 10$.





We would choose $K = 4$ clusters, which seems reasonable

General problem: the CH index is not defined for $K = 1$. We could never choose just one cluster (the null model)!

Gap statistic

It's true that $W(K)$ keeps dropping, but how much it drops at any one K should be informative

The gap statistic[4] is based on this idea. We compare the observed within-cluster variation $W(K)$ to $W_{\text{unif}}(K)$, the within-cluster variation we'd see if we instead had points distributed uniformly (over an encapsulating box). The gap for K clusters is defined as

$$\text{Gap}(K) = \log W_{\text{unif}}(K) - \log W(K)$$

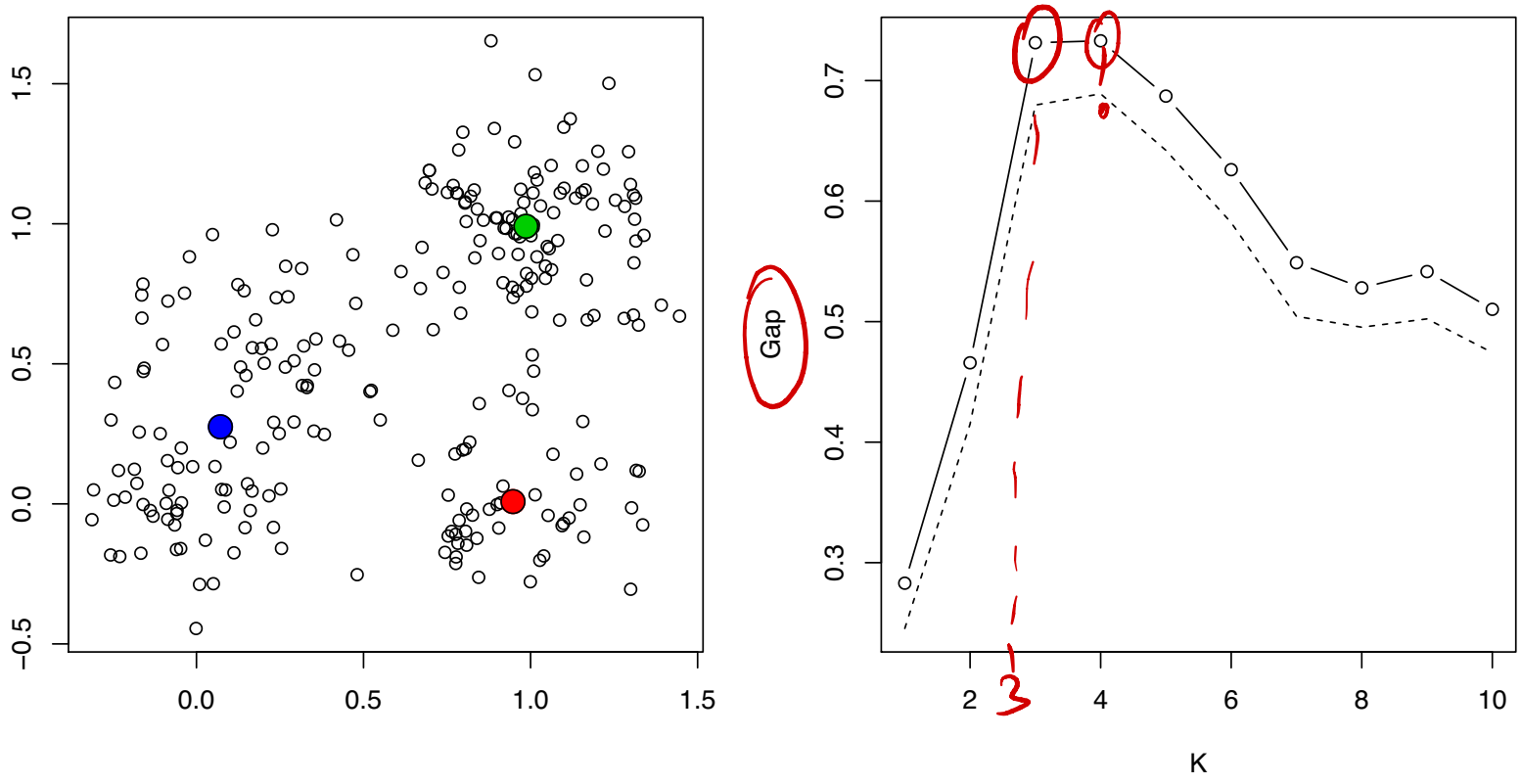
The quantity $\log W_{\text{unif}}(K)$ is computed by simulation: we average the log within-cluster variation over, say, 20 simulated uniform data sets. We also compute the standard error of $s(K)$ of $\log W_{\text{unif}}(K)$ over the simulations. Then we choose K by

$$\hat{K} = \min\left\{K \in \{1, \ldots, K_{\text{max}}\} : \text{Gap}(K) \geq \text{Gap}(K+1) - s(K+1)\right\}$$

[4] Tibshirani et al. (2001), "Estimating the number of clusters in a data set via the gap statistic"

Example: gap statistic

Running example: $n = 250$, $p = 2$, $K = 1, \ldots, 10$



We would choose $K = 3$ clusters, which is also reasonable

The gap statistic does especially well when the data fall into one cluster. (Why? Hint: think about the null distribution that it uses)

Digression: Where do (dis)similarities come from?

In our discussion of clustering, we've been assuming we have a reasonable way to measure the dissimilarity of two points

We've usually considered continuous-valued points in $\mathbb{R}^p$, where Euclidean distance didn't seem absurd.

What if our data are not all continuous, or if they are on different scales?

How can we combine these different variables into one dissimilarity?

Dissimilarities are also useful for graphical representations, dimension reduction, and even prediction/classification.

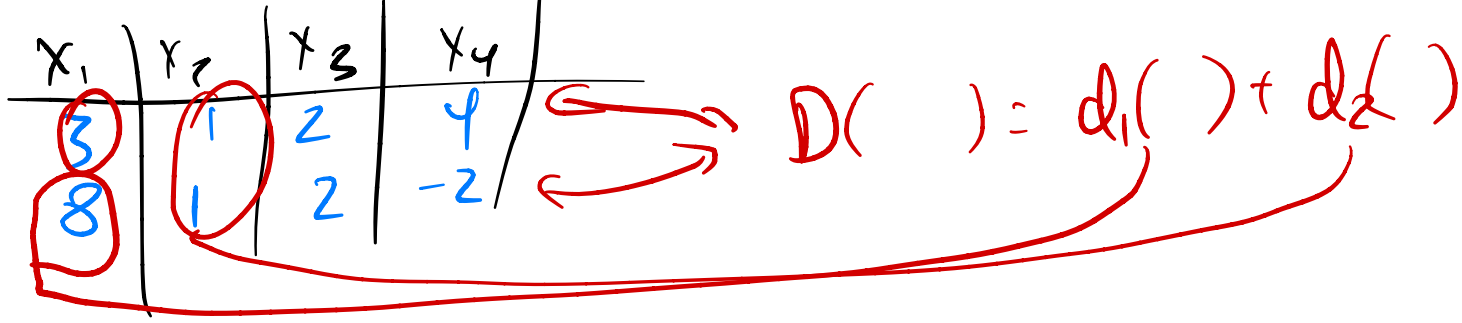
One common way of constructing dissimilarities

Suppose we have $X \in \mathbb{R}^{n \times p}$ (n observations, p variables). Each column might have different kinds of values and behaviors.

We can define a dissimilarity for each attribute, and then combine them for a dissimilarity between the rows.

$$D(x_i, x_{i'}) = \sum_{j=1}^{p} d_j(x_{ij}, x_{i'j}) \qquad \text{(Dissimilarity between rows)}$$

$d_j(x_{ij}, x_{i'j}) = ???$ (Dissimilarity for just the j^{th} attribute)



Dissimilarities for one attribute

Quantitative variables: Something that increases as $|x_{ij} - x_{i'j}|$ increases. For example, squared error: $(x_{ij} - x_{i'j})^2$. Also absolute error $|x_{ij} - x_{i'j}|$.

< Likert scale

Ordered variables: Suppose things are ranked: terrible, dislike, ok, like, wonderful. We can map them to a sequence of numbers.

Many are possible, often just into $[0, 1]$ as $\frac{i-\frac{1}{2}}{M}$ for $i = 1, \ldots, M$.

Categorical variables: Suppose we have K categories. Just like loss functions, can have a $K \times K$ matrix (usually symmetric here). For example:

	Apple	Banana	Hot pepper
Apple	0	1	10
Banana	1	0	10
Hot pepper	10	10	0

For binary data, we just wind up with $d(x_{ij}, x_{i'j}) = \mathbf{1}_{x_{ij} \neq x_{i'j}}$

Putting it together

The dissimilarities on each attribute can be combined into an overall dissimilarity, using a weighted sum.

$$D(x_i, x_{i'}) = \sum_{j=1}^{p} w_j d_j(x_{ij}, x_{i'j}) \qquad \text{with} \sum_{j=1}^{p} w_j = 1$$

Variable scale matters. If $d_1(\cdot,\cdot)$ is always in $[0,1]$, and $d_2(\cdot,\cdot)$ tends to be between 100 and 500, equal weights will make the clustering ignore variable 1! Equal influence would be given by

$$w_j \propto \left(\frac{1}{n^2} \sum_{i=1}^{n} \sum_{i'=1}^{n} d_j(x_{ij}, x_{i'j}) \right)^{-1}$$

For squared distance, this just weights by inverse sample variance.

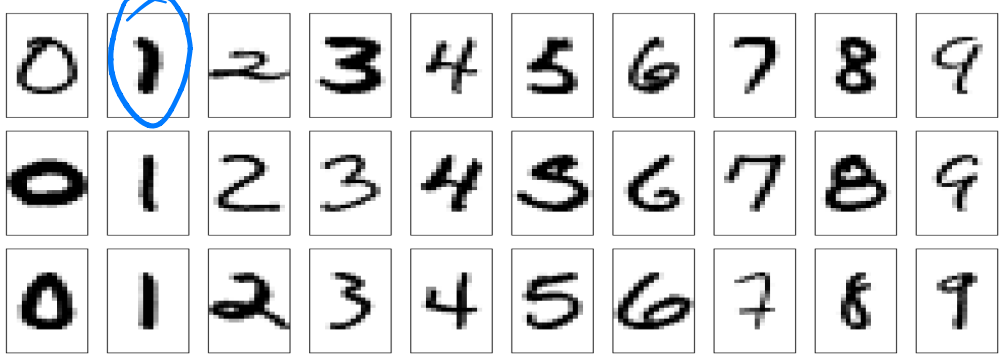
Variable importance matters. If a variable is particularly informative, giving it more weight may give better clusters than equal influence weights.

Similarities: Tangent distance

We'll take a few minutes to look at a famous example of an engineered dissimilarity. This example is described in ESL 13.3.3, and by Simard et. al. (1993).

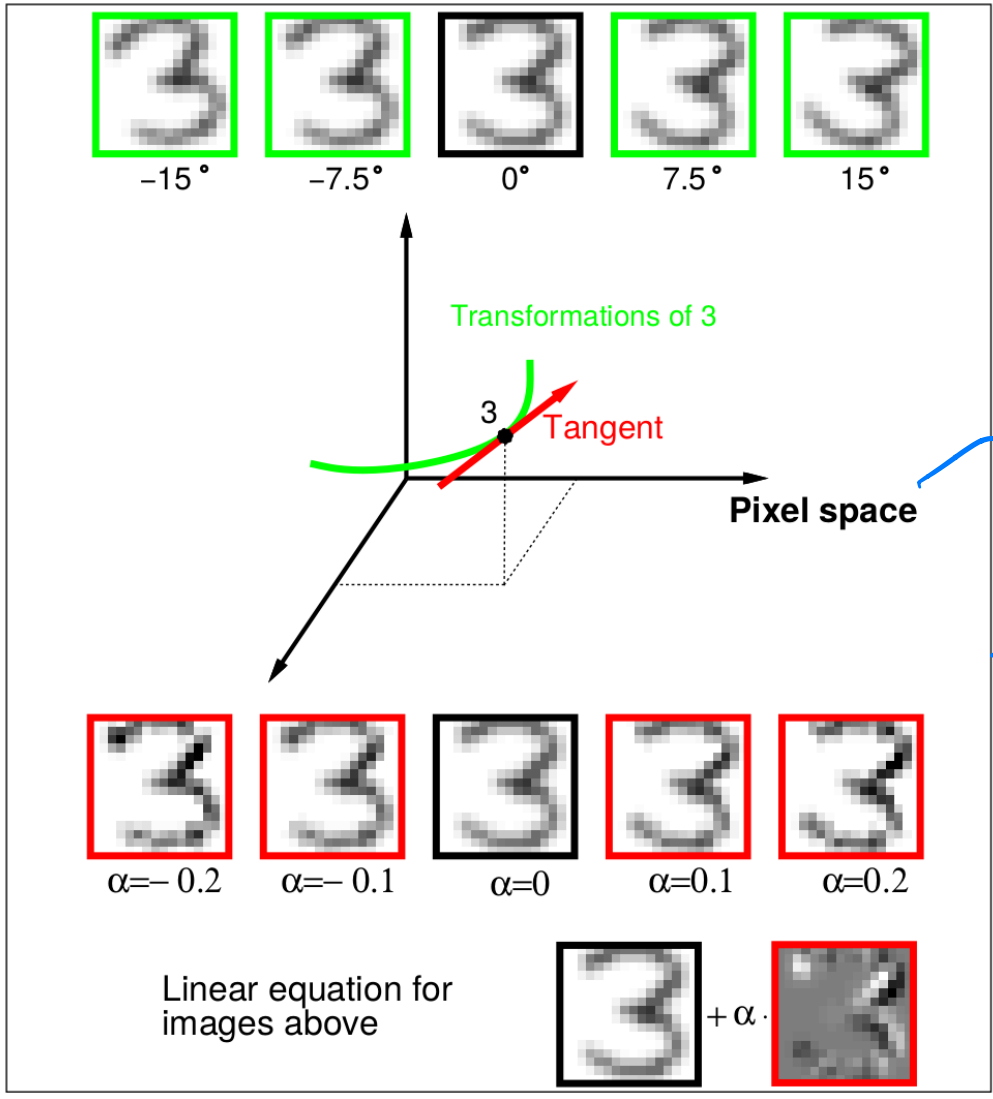
The challenge is to classify handwritten digits like those that follow. However, we know that digit recognition should be invariant to certain things: translation, rotation, shear, line thickness.

Images are 16×16, so are points in $\mathbb{R}^{256}$.



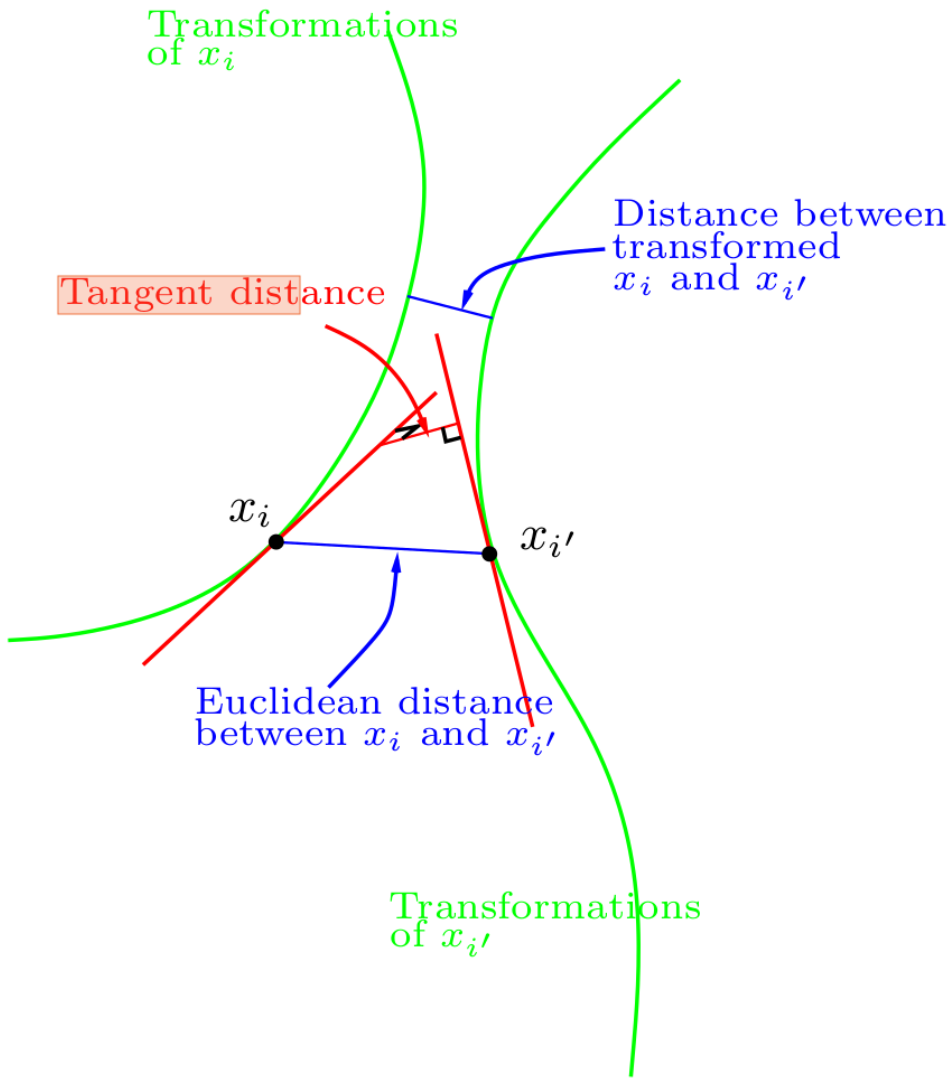
(ESL 13.9)

Similarities: Tangent distance



(ESL 13.10)

Similarities: Tangent distance



(ESL 13.11)

What is dimension reduction?

Dimension reduction: the task of transforming our data set to one with fewer features. We want this transformation to preserve the main structure that is present in the feature space

A new feature can be one of the old features, or it can be a some linear or nonlinear combination of old features.

It is often the first step in an analysis, to be followed by, e.g., visualization, clustering, regression, classification

Linear dimension reduction

We're going to start with linear dimension reduction.

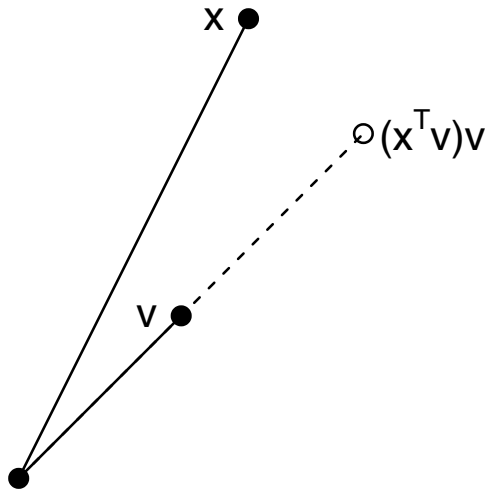
This means: looking for linear subspaces around which our data seem to concentrate.

Specifically, we'll be looking for subspaces that contain a large amount of the variance in the data.

- We hope that dimension that contain lots of the variance are also interesting...

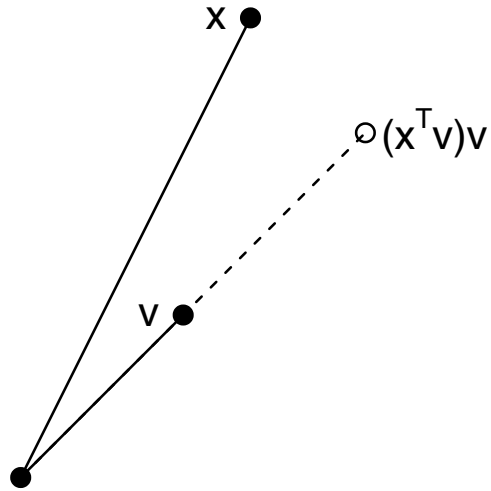
In what dimensions are things actually happening?

Review: projections onto unit vectors



A vector $v \in \mathbb{R}^p$ with $\|v\|_2^2 = v^T v = 1$ is said to have unit norm.
The projection of $x \in \mathbb{R}^p$ onto (the direction of) v is $(x^T v)v$.
Think of this as $c \cdot v$, with a coefficient or "score" of $c = x^T v$

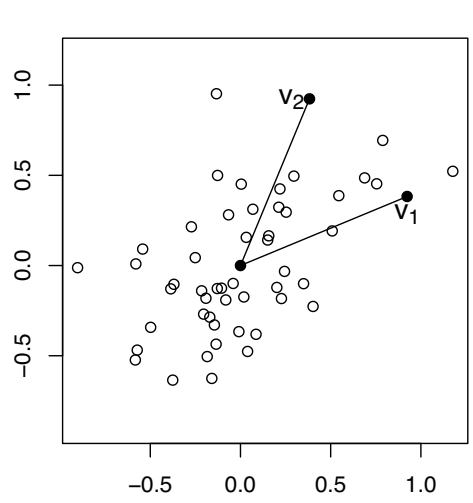
Review: projections onto unit vectors

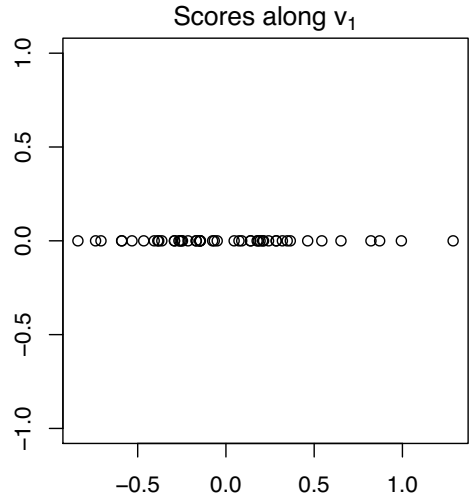


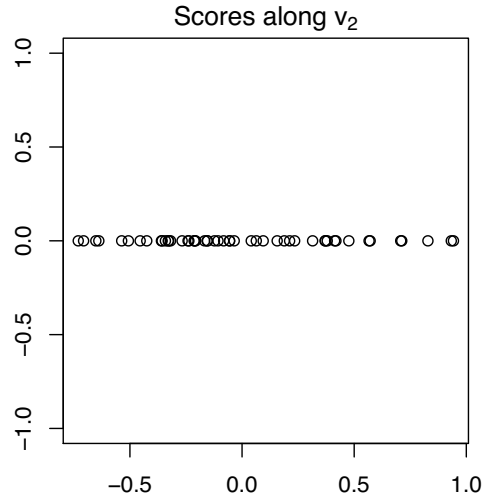
Consider a matrix $X \in \mathbb{R}^{n \times p}$. and consider projecting each row $x_i \in \mathbb{R}^p$ onto v. The entries of $Xv = \begin{pmatrix} x_1^T v \\ x_2^T v \\ \dots \\ x_n^T v \end{pmatrix} \in \mathbb{R}^n$ are the scores, and the rows of $Xvv^T \in \mathbb{R}^{n \times p}$ are the projected vectors.

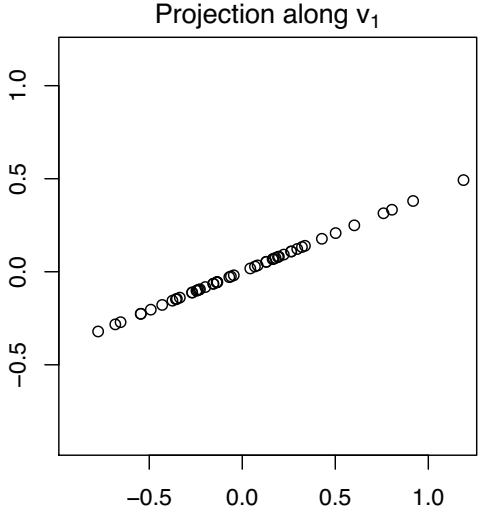
Example: projections onto unit vectors

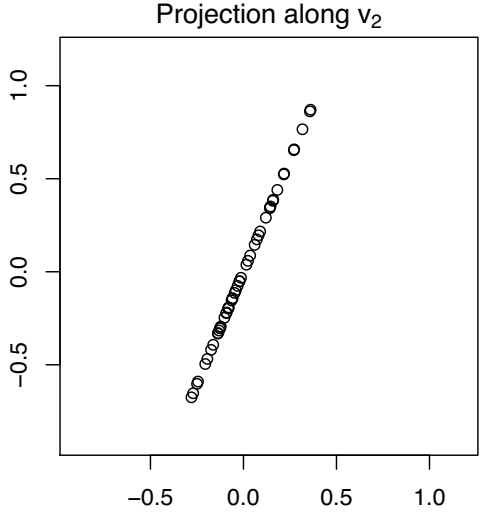
Example: $X \in \mathbb{R}^{50\times 2}$, $v_1, v_2 \in \mathbb{R}^2$











Review: projections onto orthonormal vectors

Vectors $v_1, v_2 \in \mathbb{R}^p$ are orthogonal if $v_1^T v_2 = 0$, and $v_1, \ldots v_k \in \mathbb{R}^p$ are orthogonal if $v_i^T v_j = 0$ for any i, j. Vectors $v_1, \ldots v_k \in \mathbb{R}^p$ are orthonormal if they are orthogonal and each v_j has unit norm

The projection of $x \in \mathbb{R}^p$ onto (the space spanned by) orthonormal vectors $v_1, \ldots v_k \in \mathbb{R}^p$ is $\sum_{j=1}^k (x^T v_j) v_j$. The score along the jth direction is $x^T v_j$

Review: projections onto orthonormal vectors

We now have k different vectors v_j we want to project on. Write the collection $v_1, \ldots, v_k \in \mathbb{R}^p$ as a matrix $V \in \mathbb{R}^{p \times k}$, where each v_j is a column.

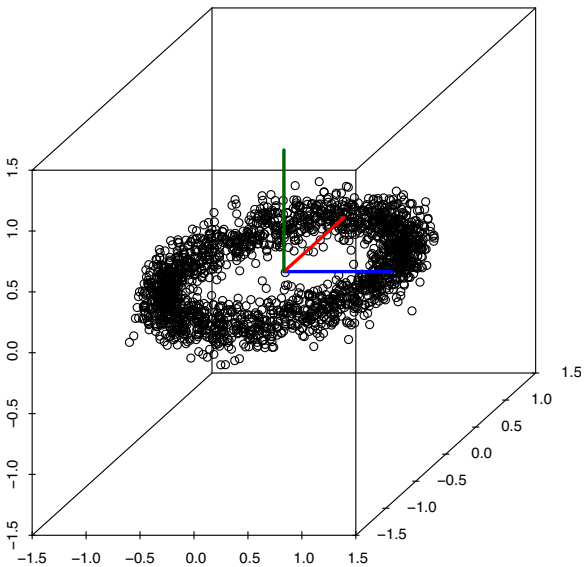
Consider a data matrix $X \in \mathbb{R}^{n \times p}$. We want to project rows of X onto columns of V.

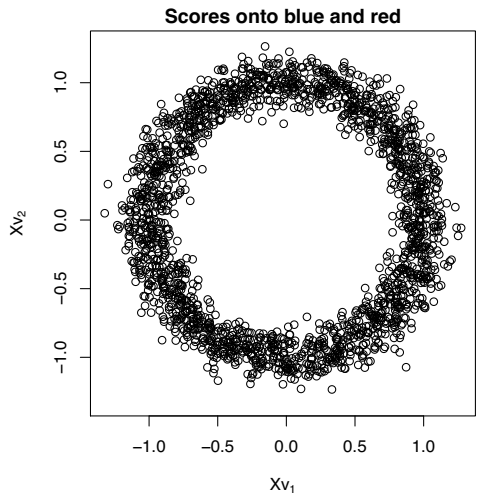
The scores are given by $XV \in \mathbb{R}^{n \times k}$, with jth column

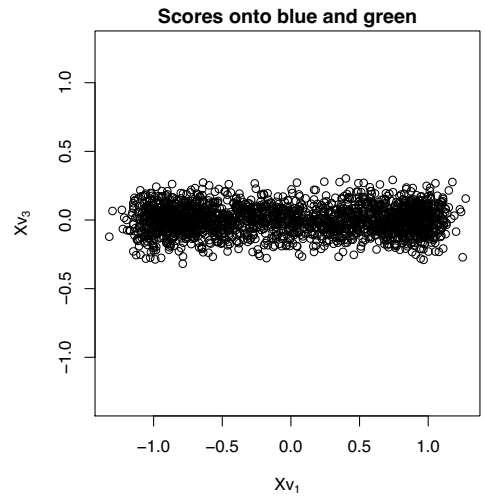
$Xv_j = \begin{pmatrix} x_1^T v_j \\ x_2^T v_j \\ \ldots \\ x_n^T v_j \end{pmatrix} \in \mathbb{R}^n$, which contains the scores from projecting X onto v_j. The projections are the rows of $XVV^T \in \mathbb{R}^{n \times p}$

Example: projections onto orthonormal vectors

Example: $X \in \mathbb{R}^{2000\times 3}$, and $v_1, v_2, v_3 \in \mathbb{R}^3$ are the unit vectors parallel to the coordinate axes





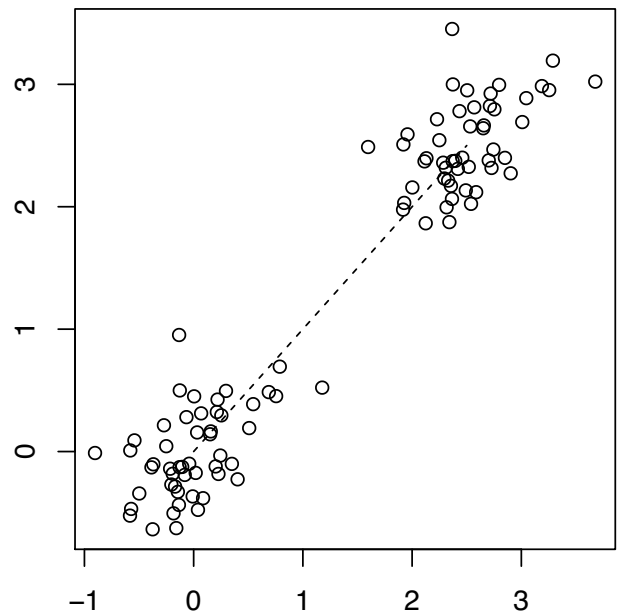


Not all linear projections are equal! What makes a good one?

Principal component analysis

Principal component analysis (PCA). You saw this in Data Science and briefly earlier this term. I just want to remind you of a few things so that we have something to build on.

We are given a data matrix $X \in \mathbb{R}^{n \times p}$, meaning that we have n observations (row vectors) and p features (column vectors). We assume that the columns of X have been centered.



First principal component direction and score

The first principal component direction of X is the unit vector $v_1 \in \mathbb{R}^p$ that maximizes the sample variance of $Xv_1 \in \mathbb{R}^n$ when compared to all other unit vectors

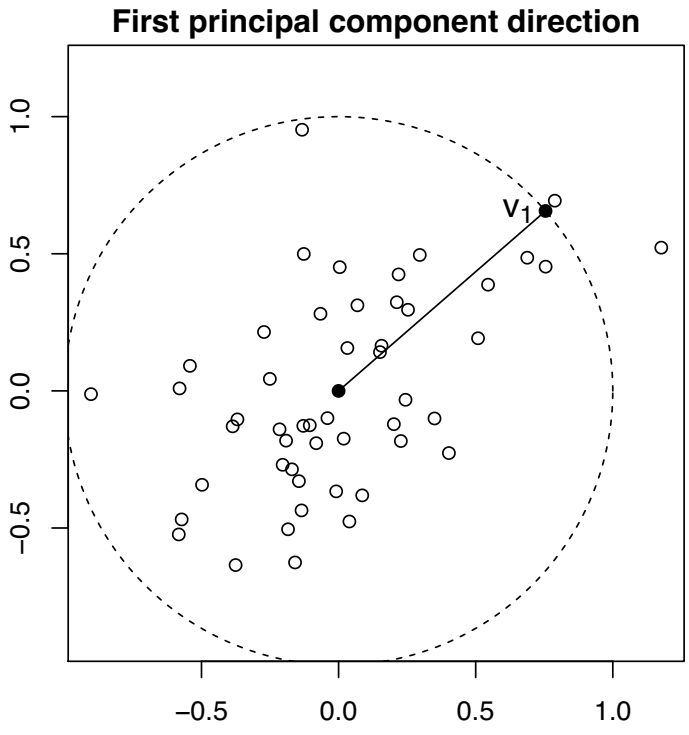
For any $v \in \mathbb{R}^p$, the vector $Xv \in \mathbb{R}^n$ has sample mean zero and sample variance $\frac{1}{n}(Xv)^T(Xv)$ (recall that we column centered X). Hence the first principal component direction $v_1 \in \mathbb{R}^p$ is

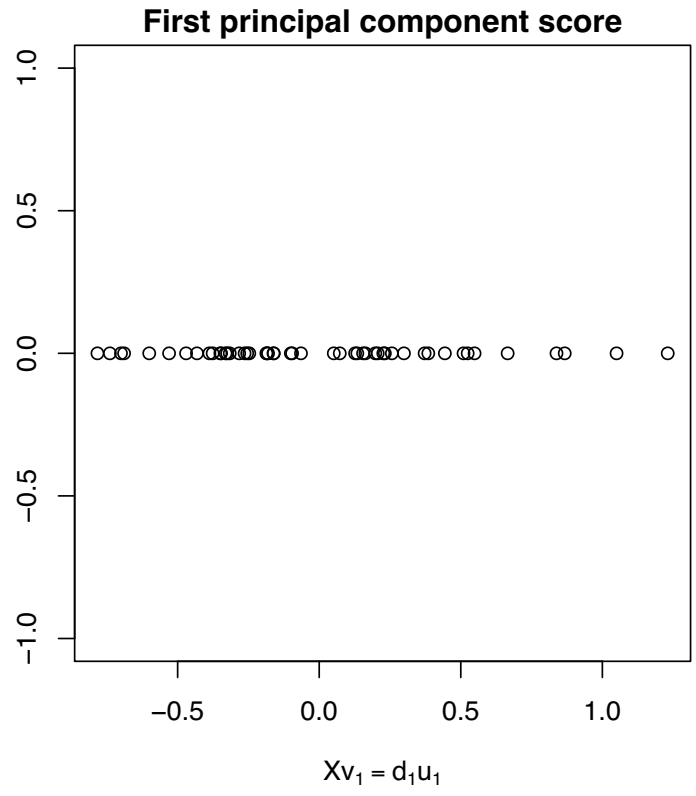
$$v_1 = \underset{\|v\|_2=1}{\operatorname{argmax}} \, (Xv)^T(Xv)$$

The vector $Xv_1 \in \mathbb{R}^n$ is called the first principal component score of X, and $u_1 = (Xv_1)/d_1 \in \mathbb{R}^n$ is the normalized first principal component score. Here $d_1 = \sqrt{(Xv_1)^T(Xv_1)}$, and d_1^2/n is the amount of variance explained by v_1

Example: first principal component direction and score

Same example data as earlier: $X \in \mathbb{R}^{50\times 2}$





Why does this agree with our eigenvalue interpretation?